ObCP

Programme modèle OBCP_template_2026

Documentation du programme — version 2.1.0

Ce document explique le fonctionnement du programme, module par module, pour qu'un débutant puisse le lire, le modifier et l'étendre : organisation, rôle de chaque fichier, chemin d'une commande, référence de toutes les fonctions, modification avec STM32CubeMX, tests sur PC, chargement par Bluetooth, exercices guidés, et ce qui a changé depuis la version 2024.

La prise en main (charger le programme, commandes) est résumée au chapitre 3 et dans le fichier README.md du projet.

Carte ObCP V2.3 — STM32L476RG, BlueNRG-M0, LIS3DH
SUPMICROTECH-ENSMM — option CROC
Joël IMBAUD — novembre 2026

Sommaire

1. Présentation 4
1.1 Le programme en bref 4
1.2 Ce qui change par rapport à la version 2024 4
1.3 Comment lire ce document 4
1.4 Le projet en un coup d'œil 5

2. La carte et les ressources utilisées 6
2.1 Brochage 6
2.2 Horloges et minuteries 7
2.3 Mémoire 7

3. Utiliser le programme 10
3.1 Charger le programme 10
3.2 Depuis un PC : le port COM virtuel 11
3.3 Depuis un téléphone : Bluetooth 11
3.4 Les commandes 11
3.5 Messages envoyés par la carte 12
3.6 Le service Bluetooth 12
3.7 Codes de la LED 13

4. Architecture du programme 14
4.1 Carte du programme 14
4.2 Code généré et code de l'application 14
4.3 La super boucle 15
4.4 Interruptions et données partagées 17
4.5 Qui appelle qui 17

5. Fonctionnement détaillé des modules 19
5.1 app.c : démarrage et boucle principale 19
5.2 commands.c : les commandes 19
5.3 line_builder.c : des caractères aux lignes 20
5.4 fifo.c : le tampon circulaire 21
5.5 usb_console.c : le port COM virtuel 21
5.6 ble_uart.c : la liaison Bluetooth 23
5.7 outputs.c : LED et sortie de puissance 25
5.8 accelerometer.c : l'accéléromètre 25
5.9 button.c : le bouton et ses rebonds 26
5.10 timer_1s.c : la minuterie d'une seconde 27
5.11 fault.c et hal_callbacks.c 27
5.12 ota_switch.c : le passage au chargeur Bluetooth 27

6. Référence des fonctions 29
6.1 app : démarrage et boucle principale 29
6.2 commands : analyse et exécution des commandes 29
6.3 outputs : LED et sortie moteur (PWM) 30
6.4 accelerometer : accéléromètre LIS3DH / LIS3DSH 30
6.5 button : bouton utilisateur 32
6.6 timer_1s : minuterie d'une seconde 32
6.7 usb_console : port COM virtuel (USB) 32
6.8 ble_uart : liaison Bluetooth 33
6.9 line_builder : assemblage des lignes de commande 35
6.10 fifo : tampon circulaire 35
6.11 fault : erreurs graves 36

6.12 hal_callbacks : rappels des interruptions ... 37
6.13 Les réglages : app_config.h ... 37
6.14 ota_switch : passage au chargeur Bluetooth ... 38
6.15 obcp_ota_request : demande au chargeur Bluetooth ... 38

7. Modifier le programme ... 39
7.1 Ajouter un traitement dans la boucle ... 39
7.2 Ajouter une commande ... 39
7.3 Envoyer des données ... 39
7.4 Changer un réglage ... 40
7.5 Modifier la configuration avec STM32CubeMX ... 40
7.6 Déboguer ... 41

8. Compiler et tester ... 42
8.1 Avec STM32CubeIDE ... 42
8.2 En ligne de commande ... 42
8.3 Tests sur PC, sans carte ... 42
8.4 Comparaison avec la version 2024 ... 43

9. Exercices guidés ... 44
9.1 Un clignotement sans attente ... 44
9.2 Une commande « c » : compteur d'appuis ... 44
9.3 Détecteur de chute ... 44
9.4 Changer le nom Bluetooth ... 44
9.5 Une broche de plus avec CubeMX ... 44

10. De la version 2024 à la version 2026 ... 45
10.1 Ce qui ne change pas ... 45
10.2 Ce que l'on peut voir ... 45
10.3 Défauts corrigés ... 45
10.4 Un choix à valider : l'échelle des LED ... 46
10.5 Correspondance des noms ... 47

Annexe A. Glossaire ... 48

Annexe B. Documentation et liens ... 49

Annexe C. Versions des outils ... 49

1. Présentation

1.1 Le programme en bref

OBCP_template_2026 est le programme modèle de la carte **ObCP** (Objet Connecté Personnel) réalisée par les étudiants de l'option CROC de SUPMICROTECH-ENSMM. Il sert de point de départ aux travaux pratiques : il fait fonctionner tout le matériel de la carte, et chacun peut y ajouter son propre code.

Le programme :

- pilote la **LED tricolore** (rouge, verte, bleue) et la **sortie de puissance du bornier J13** (moteur, 5 V), en modulation de largeur d'impulsion (PWM) ;
- lit l'**accélération** sur trois axes et la **température** de l'accéléromètre ;
- détecte les appuis sur le **bouton utilisateur** SW1 ;
- dialogue avec un **PC** par le câble USB, qui fait apparaître la carte comme un port série (port COM virtuel) ;
- dialogue avec un **téléphone** en **Bluetooth Low Energy**, par un service « UART » qui imite une liaison série ;
- comprend une douzaine de **commandes texte** (`r500`, `a`, `start timer`, `?`…), les mêmes par l'USB et par le Bluetooth ;
- **se charge par Bluetooth** (nouveau en version 2.1) : rangé derrière le chargeur Bluetooth ObCP-OTA, il reçoit un nouveau programme sans fil, envoyé depuis STM32CubeIDE ; la commande `#ota` y fait passer la carte (§ 3.1 et § 5.12).

La carte est construite autour d'un microcontrôleur **STM32L476RG** (processeur Arm Cortex-M4 à 64 MHz, 1 Mo de flash, 128 ko de RAM), d'un module Bluetooth **BlueNRG-M0** (ou SPBTLE-RF sur les cartes d'avant 2024, de la même famille BlueNRG-MS) et d'un accéléromètre **LIS3DH**. Le programme est écrit en C avec la bibliothèque HAL de ST ; sa configuration matérielle (broches, horloges, minuteries) est décrite dans le fichier `OBCP_template_2026.ioc` de STM32CubeMX.

1.2 Ce qui change par rapport à la version 2024

La version 2.0.0 remplace `OBCP_template_2024`. Elle a été réécrite entièrement, avec trois objectifs :

- **rester compatible** : mêmes commandes, mêmes réponses au caractère près, même service Bluetooth ; les scripts Python, le VI LabVIEW et les applications Android fonctionnent sans changement. Une comparaison automatique des deux versions le vérifie (§ 8.4) ;
- **corriger les défauts** de la version 2024 (plus de trente, chapitre 10) : messages perdus, débordement de tableau, valeurs mal lues, Bluetooth qui ne revenait pas après certains incidents, boucle bloquée par le bouton…
- **être lisible par un débutant** et **régénérable par STM32CubeMX** : le code est découpé en modules courts, commentés, et rangé hors des fichiers que CubeMX réécrit.

La version **2.1.0** ajoute le **chargement par Bluetooth** : le programme est rangé à `0x08080000`, derrière le chargeur Bluetooth ObCP-OTA (projet ObCP_OTA_BLE), et la nouvelle commande `#ota` fait redémarrer la carte dans ce chargeur (§ 3.1 et § 5.12). Rien d'autre ne change.

1.3 Comment lire ce document

Je veux…	Lire
utiliser la carte (charger le programme, envoyer des commandes)	chapitre 3
comprendre comment le programme est organisé	chapitres 4 et 5
connaître le rôle et les paramètres d'une fonction	chapitre 6 (référence)

Je veux…	Lire
ajouter mon code, une commande, une broche	chapitre 7, puis les exercices du chapitre 9
compiler, tester sans carte	chapitre 8
charger un programme par Bluetooth	§ 3.1 et § 5.12, puis le guide du projet ObCP_OTA_BLE
savoir ce qui a changé depuis 2024	chapitre 10
trouver la définition d'un mot (PWM, notification, NAK…)	annexe A (glossaire)

Le fichier `README.md` du projet résume l'essentiel (prise en main, commandes, corrections) ; ce document l'explique en détail. Les numéros de ligne évoluent avec le code : les fonctions sont donc citées par leur nom, et on les retrouve dans STM32CubeIDE avec *Ctrl + clic* (ou F3) sur leur nom.

1.4 Le projet en un coup d'œil

Dossier ou fichier	Contenu
`OBCP_template_2026.ioc`	configuration STM32CubeMX : broches, horloges, minuteries, USB, Bluetooth
`Core/Src`, `Core/Inc`	**l'application** (`app.c`, `commands.c`, `ble_uart.c`…) et le code généré (`main.c`, `gpio.c`, `tim.c`…)
`USB_DEVICE/`	port COM virtuel généré par CubeMX ; crochets vers l'application dans `usbd_cdc_if.c` et `usbd_conf.c`
`BlueNRG-MS/Target/`	liaison SPI avec le module Bluetooth, générée par le paquet X-CUBE-BLE1
`Drivers/`, `Middlewares/`	bibliothèques de ST : HAL, CMSIS, USB Device, BlueNRG-MS (non modifiées)
`BIN/`	programme compilé (`.hex`, `.elf`) et fichier d'installation « chargeur Bluetooth + programme » (`OBCP_template_2026_et_chargeur.hex`)
`Doc/`	ce document, les schémas (`images/`) et leurs sources (`diagrammes/`)
`Tests/`	tests sur PC et comparaison avec la version 2024 (`make`)
`Tools/verif_cubemx.py`	vérification du projet après une génération par CubeMX
`Python/`, `Labview/`	programmes PC d'exemple (inchangés)
`Datasheets/`	documentation des composants, procédure de programmation
`Makefile`	compilation en ligne de commande, sans STM32CubeIDE (facultatif)
`STM32L476RGTX_FLASH.ld`	script d'édition de liens : le programme commence à `0x08080000`, derrière le chargeur Bluetooth

2. La carte et les ressources utilisées

2.1 Brochage

La figure 1 montre les broches du STM32L476RG utilisées par le programme. Chaque broche a dans CubeMX un **nom** (*User Label*) que le code utilise à la place de son numéro : CubeMX en tire les définitions `LED_RED_Pin`, `LED_RED_GPIO_Port`… de `main.h`. Changer un nom dans CubeMX oblige donc à changer le code (l'outil `verif_cubemx.py` le signale).

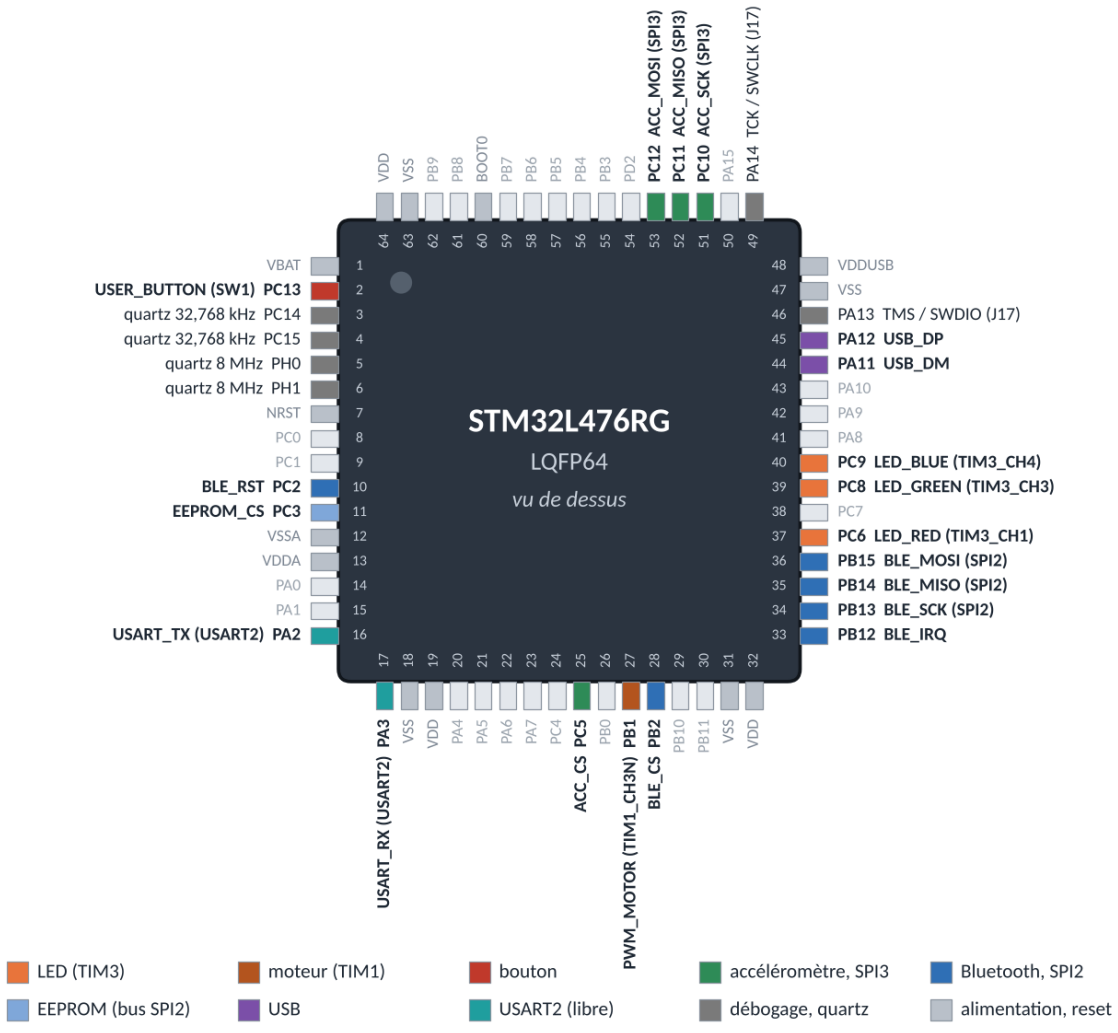


Figure 1 – *broches du STM32L476RG utilisées par le programme (boîtier LQFP64)*

Fonction	Broches	Périphérique	Nom (CubeMX)
LED rouge, verte, bleue (D5 ; transistors Q1 à Q3)	PC6, PC8, PC9	TIM3 voies 1, 3, 4	`LED_RED`, `LED_GREEN`, `LED_BLUE`
sortie de puissance J13 (MOSFET M1, 5 V)	PB1	TIM1 voie 3N	`PWM_MOTOR`
bouton utilisateur SW1 (rappel R1 à 3,3 V)	PC13	EXTI13, front descendant	`USER_BUTTON`
accéléromètre U4	PC10, PC11, PC12 ; CS = PC5	SPI3	`ACC_SCK`, `ACC_MISO`, `ACC_MOSI`, `ACC_CS`

Fonction	Broches	Périphérique	Nom (CubeMX)
module Bluetooth BLE1	PB13, PB14, PB15 ; CS = PB2 ; IRQ = PB12 ; reset = PC2	SPI2, EXTI12	`BLE_SCK`, `BLE_MISO`, `BLE_MOSI`, `BLE_CS`, `BLE_IRQ`, `BLE_RST`
mémoire EEPROM U7 (M95640, 64 kbit), sur le bus du module	CS = PC3	SPI2 (non utilisée)	`EEPROM_CS`
port COM virtuel	PA11, PA12	USB OTG FS, classe CDC	—
liaison série libre (J5, J7)	PA2, PA3	USART2, 115200 bauds (non utilisée)	`USART_TX`, `USART_RX`
débogage (connecteur J17)	PA13, PA14	SWD	`TMS`, `TCK`

> **Bon à savoir**
>
> Les deux broches de sélection **ACC_CS** (PC5) et **BLE_CS** (PB2), ainsi que **EEPROM_CS** (PC3), démarrent à 1 : à 0, elles sélectionnent leur circuit. Le module Bluetooth et l'EEPROM partagent le bus SPI2 : une seule sélection à la fois.
>
> La nomenclature de la carte indique un accéléromètre LIS3DSH, mais la liste d'achat et la fiche fournie correspondent au **LIS3DH** (même boîtier, registres différents). Le programme lit le registre d'identité du capteur au démarrage et sait piloter les deux (§ 5.8).

2.2 Horloges et minuteries

Le quartz de 8 MHz (HSE) alimente une boucle à verrouillage de phase (PLL) qui fournit **64 MHz** au processeur et aux minuteries ; une seconde PLL (PLLSAI1) fournit les 48 MHz de l'USB. Une minuterie compte au rythme de l'horloge divisée par (*Prescaler* + 1), de 0 jusqu'à *Period* (registre ARR), puis repart de 0. En PWM, la sortie est à 1 tant que le compteur est inférieur à la valeur de comparaison (registre CCR) :

```
fréquence PWM   = 64 MHz / ((Prescaler + 1) x (Period + 1))
rapport cyclique = CCR / (Period + 1)
```

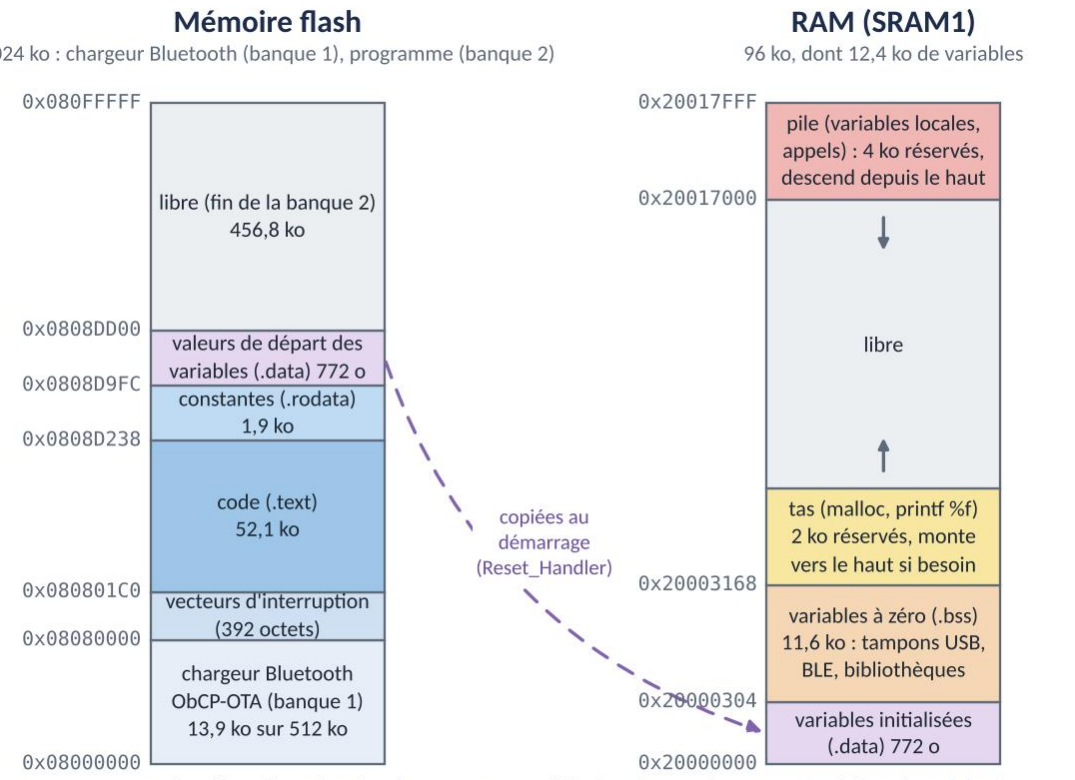
Minuterie	Rôle	Prescaler	Period	Résultat
TIM3	LED (voies 1, 3, 4)	0	65535	PWM à 976,6 Hz ; rapport cyclique = n / 65536 pour `rn`
TIM1	sortie J13 (voie 3N)	0	65535	PWM à 976,6 Hz ; rapport cyclique = n / 65536 pour `mn`
TIM7	minuterie de `start timer`	6399	9999	64 MHz / 6400 / 10 000 = 1 interruption par seconde
SysTick	base de temps de la HAL	—	—	1 interruption par milliseconde : `HAL_GetTick()`, `HAL_Delay()`

Avec 65536 pas, une commande `rn` règle la LED de façon progressive sur toute l'échelle 0 à 65535 annoncée par l'aide. En 2024, TIM3 comptait jusqu'à 6399 : la LED était à pleine lumière dès `r6399` (voir § 10.4).

2.3 Mémoire

Le programme compilé se range dans deux mémoires (figure 2) :

- la **flash** (1 Mo, conservée hors tension) contient le code (`.text`), les constantes (`.rodata` : textes des messages, aide…) et les valeurs de départ des variables initialisées ;
- la **RAM** (ici les 96 ko de la SRAM1) contient les variables : celles qui ont une valeur de départ (`.data`, recopiées depuis la flash au démarrage), celles qui partent de zéro (`.bss`), le **tas** (mémoire demandée par `malloc`, utilisée par `printf` pour afficher les nombres à virgule) et la **pile** (variables locales des fonctions, adresses de retour).



Hauteurs non proportionnelles. Adresses lues dans le programme compilé (Release). SRAM2 (0x10000000, 32 ko) : seuls ses 8 derniers octets (0x10007FF0) servent, de boîte aux lettres entre le programme et le chargeur (commande #ota).

Figure 2 – *plan de la mémoire du programme compilé (version Release)*

Depuis la version 2.1, la flash est partagée en deux moitiés : la première (banque 1, à partir de `0x08000000`) est réservée au **chargeur Bluetooth** ObCP-OTA, et le programme commence à `0x08080000` (banque 2, 512 ko). Cette adresse est fixée par la ligne `FLASH` du script d'édition de liens `STM32L476RGTX_FLASH.ld`. Huit octets de la SRAM2 (à `0x10007FF0`) servent de « boîte aux lettres » entre le programme et le chargeur (§ 5.12) ; le programme n'utilise pas la SRAM2 autrement.

Le programme occupe 55,1 ko de flash, sur les 512 ko de la banque 2, et 12,3 ko de RAM sur 96, plus 6 ko réservés au tas (2 ko) et à la pile (4 ko). La figure 3 détaille ces tailles par partie du programme : l'application elle-même est petite ; la bibliothèque C (surtout l'affichage des nombres à virgule par `printf`) et la HAL pèsent davantage. En RAM, les plus gros postes sont les tampons de l'USB (2 × 2 ko dans le code généré par CubeMX, 1 ko de file d'envoi dans l'application) et la file d'envoi du Bluetooth (1 ko).

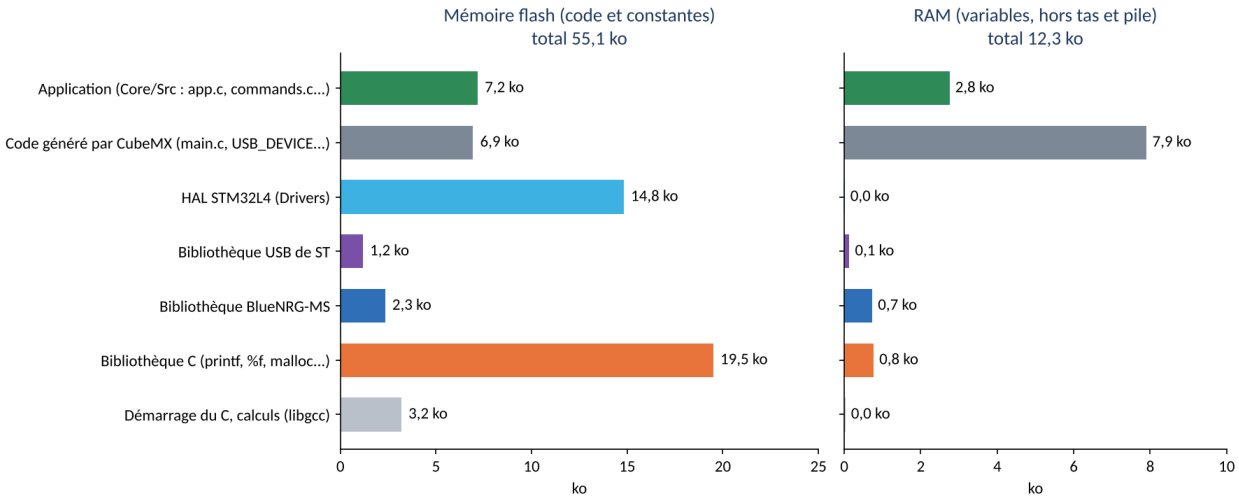


Figure 3 – *occupation de la mémoire par partie du programme (lue dans le fichier .map de l'édition de liens)*

Taille de chaque module de l'application, en octets (code, constantes, variables initialisées, variables à zéro) :

Module	Code	Constantes	.data	.bss
ble_uart.c	2088	58	18	1349
commands.c	810	1136	0	0
usb_console.c	864	0	12	1376
app.c	530	213	0	2
accelerometer.c	608	0	12	21
fault.c	256	0	0	0
ota_switch.c	224	0	0	5
line_builder.c	224	0	0	0
fifo.c	192	0	0	0
outputs.c	148	32	0	8
button.c	176	0	0	6
timer_1s.c	148	0	0	9
hal_callbacks.c	32	0	0	0

Les 1 ko de constantes de `commands.c` sont surtout le texte de l'aide ; les tableaux de `.bss` de `usb_console.c` et `ble_uart.c` sont leurs files d'envoi.

3. Utiliser le programme

3.1 Charger le programme

Depuis la version 2.1, le programme est rangé derrière le **chargeur Bluetooth** ObCP-OTA (§ 2.3) : à chaque démarrage, le chargeur lance le programme en quelques millisecondes ; il ne reste actif que si on le lui demande (figure 4).

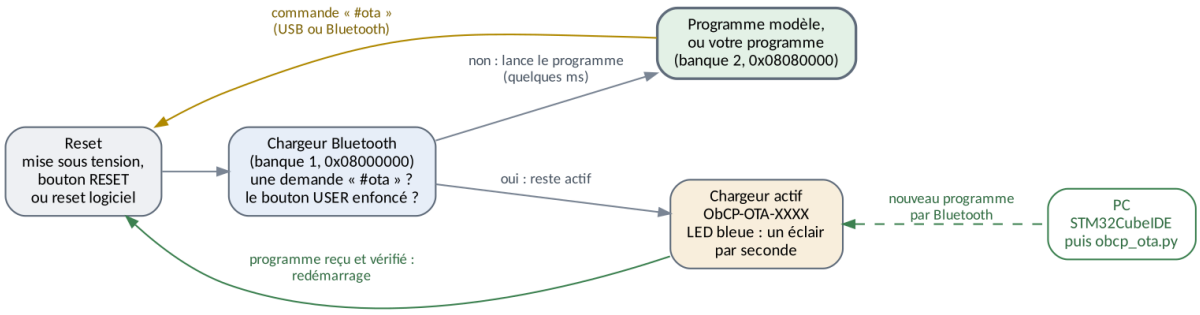


Figure 4 – *chargement par Bluetooth : le chargeur, le programme et la commande #ota*

Première installation, une seule fois par carte : le chargeur et le programme, par le câble USB (mode DFU, *Device Firmware Upgrade*, géré par le chargeur intégré au STM32), avec STM32CubeProgrammer :

1. brancher l'ObCP au PC ; interrupteur **SW4 sur « Prog »**, **SW2 sur « On »**, appui sur **SW3** (reset) ;
2. dans STM32CubeProgrammer : choisir **USB** en haut à droite, cliquer sur le bouton de rafraîchissement, puis sur **Connect** ;
3. menu **Erasing & programming** : *Browse* → `BIN/OBCP_template_2026_et_chargeur.hex` (un fichier `.hex` contient ses adresses : rien à saisir), puis **Start Programming** ;
4. **SW4 sur « Run »**, appui sur **SW3** : le programme démarre, la LED fait un éclat vert.

Un fichier `.bin` chargé à `0x08000000`, comme dans l'ancienne procédure (illustrée dans `Datasheets/Procedure_Programmation_OBCP_CROC.pdf`), effacerait le chargeur ; et le programme, prévu pour `0x08080000`, ne fonctionnerait pas à cette adresse. Avec un `.hex`, l'adresse n'est pas demandée.

Ensuite, par Bluetooth, avec l'application PC `obcp_ota.py` du projet ObCP_OTA_BLE (Python 3 et la bibliothèque Bleak) : dans STM32CubeIDE, sélectionner le projet puis *Run > External Tools > ObCP OTA - compiler et transferer* ; ou dans un terminal : `python obcp_ota.py ...\Debug\OBCP_template_2026.elf`. Pour recevoir un programme, la carte doit être dans le chargeur :

- `obcp_ota.py` (version 1.2 ou plus récente, sous Windows ou Linux) y met lui-même une carte qui a déjà reçu un programme depuis ce PC avec cette version : il a mémorisé son adresse Bluetooth et lui envoie `#ota` ;
- la commande `#ota`, depuis le terminal ou le téléphone (§ 3.4) ;
- le bouton **USER (SW1)** maintenu enfoncé pendant un appui sur **RESET (SW3)**.

Dans le chargeur, la LED bleue émet un bref éclair chaque seconde (§ 3.7), le port COM virtuel disparaît du PC (le chargeur n'utilise pas l'USB) et la carte se présente en Bluetooth sous le nom `ObCP-OTA-XXXX`. Sans connexion pendant 2 minutes, ou après un appui sur RESET, le chargeur relance le programme. Le guide du projet ObCP_OTA_BLE détaille l'installation de l'application PC et ses messages.

Sans le chargeur (programme seul, comme en version 2.0) : remettre dans `STM32L476RGTX_FLASH.ld` la ligne `FLASH (rx) : ORIGIN = 0x8000000, LENGTH = 1024K`, recompiler et charger le `.bin` à `0x08000000` ; `#ota` ne fait alors que redémarrer la carte. Une sonde ST-LINK (celle d'une carte Nucleo, par exemple) branchée sur le connecteur J17 permet aussi de programmer et de déboguer depuis STM32CubeIDE (§ 7.6).

3.2 Depuis un PC : le port COM virtuel

Branchée par USB, la carte se présente comme un port série : « Périphérique série USB (COMx) » dans le gestionnaire de périphériques de Windows, `/dev/ttyACM0` sous Linux. On l'ouvre avec un terminal (PuTTY, Tera Term…), avec les scripts du dossier `Python/` (bibliothèques `pyserial` et `matplotlib`) ou avec le VI LabVIEW du dossier `Labview/`.

- **La vitesse (bauds) choisie n'a aucun effet** : les données voyagent à la vitesse de l'USB.
- **Fin de commande** : CR, LF ou CR+LF, au choix ; la touche d'effacement (Backspace) corrige la saisie.
- Un programme qui envoie une commande d'un seul bloc peut omettre la fin de ligne (`r500` seul) ; un caractère envoyé seul est traité comme une frappe au clavier, et attend la touche Entrée (§ 5.5).

3.3 Depuis un téléphone : Bluetooth

La carte se signale sous le nom **OBCP-CROC**. Deux applications Android accompagnent l'ObCP (dossier `Android/` du dépôt LAB_ObCP_L476_STM32CubeIDE) : **ObCP Control** (curseurs des LED et de la sortie PWM, courbes d'accélération et de température) et **BLE Terminal JILAB** (terminal texte). Une application « UART BLE » générique convient aussi, à condition de désigner les bonnes caractéristiques (§ 3.6). Si le téléphone demande un code d'appairage : **123456** ; aucun appairage n'est nécessaire en temps normal.

3.4 Les commandes

Commande	Effet	Réponse
`r`*n* ou `R`*n*	LED **rouge**, *n* de 0 (éteinte) à 65535 (pleine lumière)	`Red = n`
`v`*n* ou `V`*n*	LED **verte**	`Green = n`
`b`*n* ou `B`*n*	LED **bleue**	`Blue = n`
`m`*n* ou `M`*n*	sortie de puissance **J13** (moteur)	`PWM = n`
`a` ou `A`	lit l'**accéléromètre** (en g)	`Acceleration : X=0.01 Y=-0.02 Z=1.00`
`t` ou `T`	lit la **température** du capteur (°C, ordre de grandeur)	`T = 27`
`?`	**aide**, envoyée seulement à l'interface qui la demande	liste des commandes
`cls`	**efface l'écran** du terminal (USB seulement)	séquence ANSI `ESC[1;1H ESC[2J`
`start timer`	envoie au téléphone, **chaque seconde**, l'accélération et la température	aucune
`stop timer`	arrête cet envoi	aucune
`#rst`	**tout éteint** (LED, sortie J13) et arrête la minuterie	aucune
`#ota`	**redémarre dans le chargeur Bluetooth** (nouveau en 2.1) : tout s'éteint, la réponse part, la connexion Bluetooth se ferme, puis la carte redémarre (§ 5.12)	`OTA loader...`

Règles communes :

- toutes les réponses se terminent par `\n\r` (saut de ligne puis retour chariot), comme en 2024 ;
- une réponse est aussi **recopiée sur l'autre interface** si elle est active (USB reconnu par le PC, téléphone connecté) : on voit dans le terminal les commandes du téléphone, et inversement ; l'aide n'est pas recopiée ;
- *n* est un nombre **décimal** de 0 à 65535, éventuellement précédé d'espaces (`r 500`) ;
- les espaces au début et à la fin de la ligne sont ignorés, une ligne vide aussi ; `start timer`, `stop timer`, `#rst`, `#ota` et `cls` s'écrivent en minuscules ;

- erreurs : `False command` (commande inconnue, ligne de plus de 64 caractères, `cls` par Bluetooth), `False value (0-65535)` (valeur absente, négative, trop grande, ou suivie d'autre chose : `r`, `r-1`, `r70000`, `r50x`), `Accelerometer error` (capteur absent ou muet).

Exemple de session dans un terminal :

```
r20000          ->  Red = 20000
v 500           ->  Green = 500
a               ->  Acceleration : X=0.01 Y=-0.02 Z=1.00
t               ->  T = 27
r70000          ->  False value (0-65535)
hello           ->  False command
#rst            ->  (pas de réponse : tout est éteint)
#ota            ->  OTA loader...   (puis la carte redémarre dans le chargeur)
```

3.5 Messages envoyés par la carte

Quand	Vers le terminal (USB)	Vers le téléphone (BLE)
un téléphone se connecte	`Generated BLE Address: C1:23:45:67:89:AB` (carte) `Connected to device: …` (téléphone)	—
le téléphone se déconnecte : LED et sortie J13 s'éteignent, la minuterie s'arrête ; aussi après `#ota`, quand la carte ferme elle-même la connexion	`BLE disconnected :p`	—
l'USB et le Bluetooth sont connectés tous les deux (une fois après chaque branchement du câble)	—	`USB Connected`
appui sur le bouton SW1	`Push button pressed`	`PButton pressed`
chaque seconde après `start timer`	—	`X0.01Y-0.02Z1.00` puis `T = 27`

Chaque message part dans sa propre notification Bluetooth : l'application ObCP Control reconnaît ainsi les mesures, qui commencent par `X` ou par `T =` .

3.6 Le service Bluetooth

Élément	UUID	Propriétés	Sens
service « UART »	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	—	—
TX	`6E400002-B5A3-F393-E0A9-E50E24DCCA9E`	notification, lecture	carte → téléphone
RX	`6E400003-B5A3-F393-E0A9-E50E24DCCA9E`	écriture, écriture sans réponse	téléphone → carte

- Ce sont les UUID du *Nordic UART Service* (NUS), mais **avec TX et RX inversés** par rapport à la norme Nordic (chez Nordic, `…02` reçoit et `…03` émet). C'était déjà le cas en 2024 et l'application ObCP Control en dépend : ne pas le changer. Avec nRF Connect : écrire dans `…03`, s'abonner aux notifications de `…02`.
- Une notification transporte **20 octets au plus** (taille de base d'un paquet BLE) : un message plus long est découpé en plusieurs notifications ; une commande écrite dans RX fait elle aussi 20 octets au plus.
- L'adresse de la carte est l'adresse « statique aléatoire » propre à chaque module, comme en 2024 (§ 5.6) ; elle est affichée dans le terminal à chaque connexion.
- Le chargeur Bluetooth utilise une adresse voisine, dont le premier octet affiché diffère d'une unité (`C0:…` pour `C1:…`), et le nom `ObCP-OTA-XXXX` (XXXX : fin de l'adresse) : le PC voit deux appareils distincts. C'est ainsi que `obcp_ota.py` reconnaît le programme d'une carte dont il connaît le chargeur.

- Publicité toutes les 1,28 à 2,56 s ; puissance d'émission d'environ −2 dBm.

3.7 Codes de la LED

Au démarrage, un seul signal, puis le programme fonctionne normalement :

Signal	Signification
1 éclat vert	tout est prêt
2 éclats rouges	accéléromètre absent ou muet : `a` et `t` répondent `Accelerometer error`
3 éclats rouges	module Bluetooth absent ou muet : l'USB fonctionne
2 éclats rouges, pause, 3 éclats rouges	les deux

Erreur grave : la LED rouge clignote sans fin (N éclats, une pause, N éclats…), LED et sortie J13 coupées ; seul un reset (SW3) fait repartir le programme :

Éclats	Cause
1	`Error_Handler()` : une initialisation générée par CubeMX a échoué (horloge, SPI, USB…)
2	HardFault : faute grave qu'aucun gestionnaire plus précis n'a pu traiter
3	MemManage : exécution de code dans une zone interdite
4	BusFault : lecture ou écriture à une adresse où il n'y a rien (pointeur jamais initialisé…)
5	UsageFault : instruction invalide, appel d'un pointeur de fonction NULL, division entière par zéro
6	NMI : interruption non masquable inattendue

Dans le chargeur Bluetooth (après `#ota`, ou USER maintenu pendant un appui sur RESET) :

Signal	Signification
bleu, bref éclair chaque seconde	attente d'une connexion
violet, bref éclair chaque seconde	attente, et aucun programme valide en banque 2
bleu fixe	PC connecté
vert qui change à chaque ko reçu	transfert en cours
vert fixe	programme reçu, vérifié et validé : il démarre ensuite
rouge fixe	erreur de transfert (explication sur le PC)
rouge, clignotement rapide	le module Bluetooth ne répond pas
blanc clignotant (3 s)	identification demandée par le PC

4. Architecture du programme

4.1 Carte du programme

Le programme est organisé en trois couches (figure 5) :

- en haut, le **code généré par STM32CubeMX** (`main.c`, `gpio.c`, `tim.c`, `usbd_cdc_if.c`…), qui configure le matériel d'après le fichier `.ioc` ;
- au milieu, **l'application** : une douzaine de modules (un fichier `.c` et un fichier `.h` chacun) écrits pour l'ObCP ;
- en bas, les **bibliothèques de ST** : la HAL (accès aux périphériques), la bibliothèque USB Device et la bibliothèque BlueNRG-MS (dialogue avec le module Bluetooth).

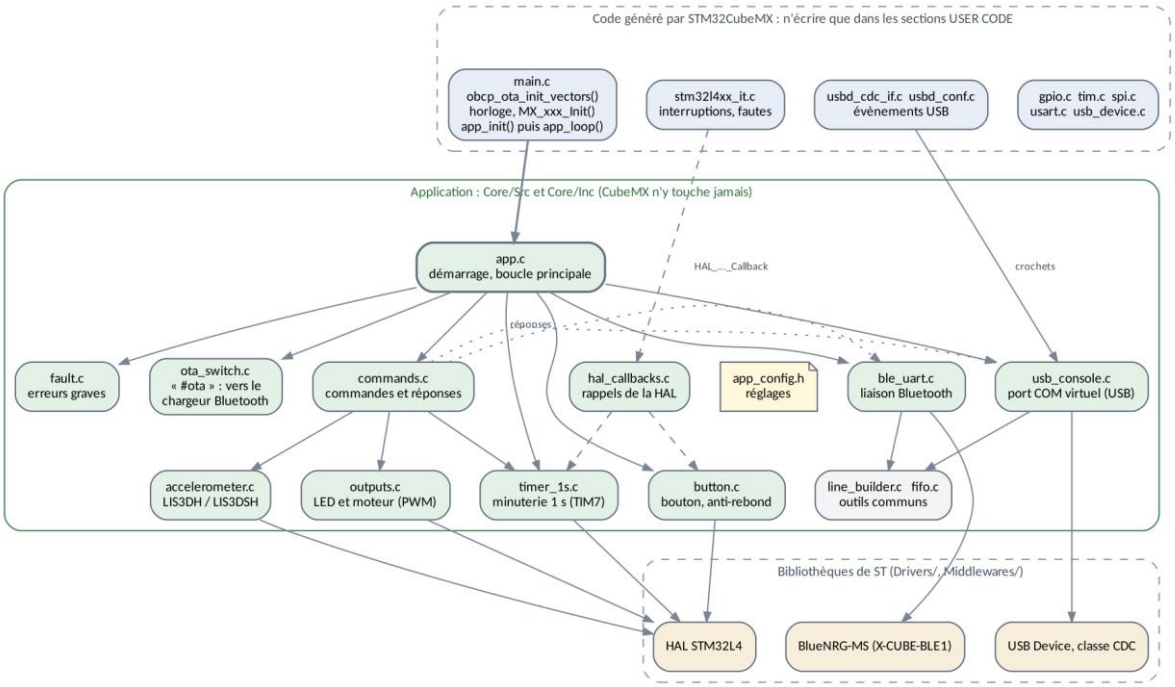


Figure 5 – *organisation du programme : code généré, application, bibliothèques*

Chaque module de l'application a **une seule responsabilité** et s'utilise par les fonctions déclarées dans son `.h` ; ses variables sont `static` (invisibles des autres fichiers). Par exemple, `commands.c` ne sait pas comment une LED est branchée : il appelle `outputs_set(OUTPUT_RED, n)`, et seul `outputs.c` connaît TIM3. Les modules du bas (`outputs`, `accelerometer`, `button`, `timer_1s`, `line_builder`, `fifo`) n'appellent aucun autre module : on peut les comprendre, les tester ou les réutiliser seuls.

4.2 Code généré et code de l'application

STM32CubeMX réécrit ses fichiers à chaque génération du code, **sauf le texte placé entre les commentaires** `/* USER CODE BEGIN xxx */` et `/* USER CODE END xxx */`. Le programme en tient compte :

- tout le code de l'application est dans **ses propres fichiers** (`Core/Src/app.c`, `commands.c`…), que CubeMX ne connaît pas et ne touche jamais ;
- les fichiers générés ne contiennent que quelques **lignes « crochets »**, toujours dans une section USER CODE, qui passent la main à l'application.

Fichier généré	Section USER CODE	Crochet	Rôle
`main.c`	Includes	`#include "app.h", #include "fault.h", #include "obcp_ota_request.h"`	déclarations

Fichier généré	Section USER CODE	Crochet	Rôle
`main.c`	1	`obcp_ota_init_vectors();`	table des interruptions du programme, rangé derrière le chargeur Bluetooth (§ 5.12)
`main.c`	2	`app_init();`	démarrage de l'application
`main.c`	3 (dans `while (1)`)	`app_loop();`	un tour de la boucle principale
`main.c`	Error_Handler_Debug	`fault_signal(FAULT_ERROR_HANDLER);`	erreur d'initialisation
`stm32l4xx_it.c`	xxx_IRQn 0 des 5 fautes	`fault_signal(FAULT_HARDFAULT);…`	fautes du processeur
`usbd_cdc_if.c`	3, 4	`usb_console_on_init();`, `usb_console_on_deinit();`	liaison USB ouverte, fermée
`usbd_cdc_if.c`	6	`usb_console_on_receive(Buf, *Len);`	paquet reçu du PC
`usbd_cdc_if.c`	7	`usb_console_write(Buf, Len)`	`CDC_Transmit_FS()` passe par la file d'envoi
`usbd_conf.c`	2, 3	`usb_console_on_suspend();`, `usb_console_on_resume();`	veille et réveil du bus USB

Les interruptions du bouton et de TIM7 n'ont pas besoin de crochet : la HAL appelle d'elle-même `HAL_GPIO_EXTI_Callback()` et `HAL_TIM_PeriodElapsedCallback()`, que l'application écrit dans `hal_callbacks.c` (§ 4.4). L'outil `Tools/verif_cubemx.py` vérifie tous ces crochets après une génération (§ 7.5).

4.3 La super boucle

Le programme n'a pas de système d'exploitation : après le démarrage, `main()` appelle `app_loop()` sans fin. Ce modèle, très courant sur les petits microcontrôleurs, s'appelle une **super boucle** (*super loop*, ou programmation *bare metal*). Chaque tour regarde ce qui est arrivé depuis le tour précédent et fait le travail correspondant (figure 6).

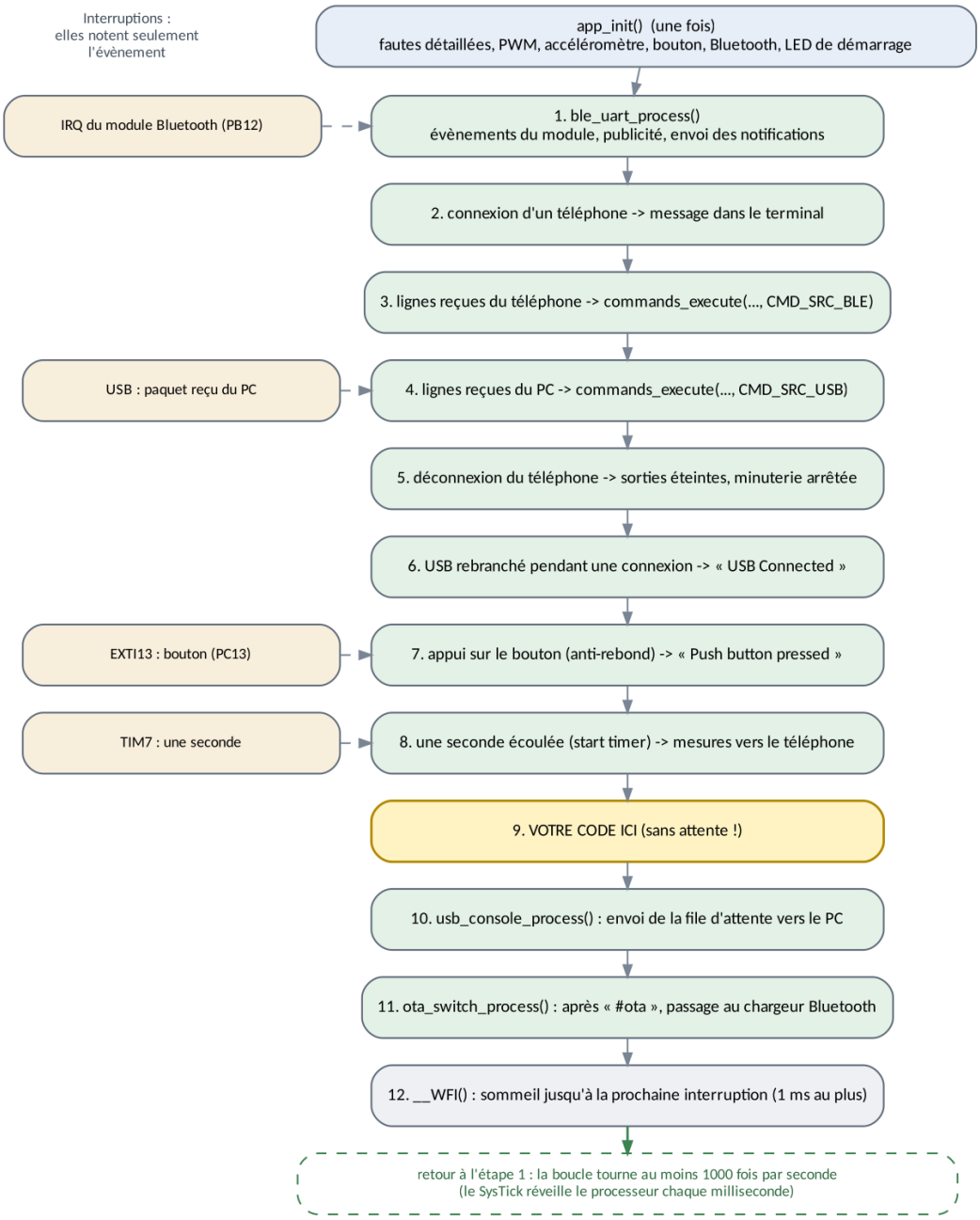


Figure 6 – *un tour de la boucle principale app_loop() et les interruptions qui l'alimentent*

Deux règles font fonctionner la super boucle :

1. **aucune étape n'attend.** Une attente (`HAL_Delay()`, boucle qui guette un évènement) bloquerait toutes les autres : pendant ce temps, l'USB et le Bluetooth ne seraient plus servis, et les messages s'accumuleraient. Pour agir plus tard, on note l'instant (`HAL_GetTick()`) et on revient voir aux tours suivants ;
2. **les interruptions ne font que noter l'évènement** (un drapeau, un paquet mis de côté) et rendent la main aussitôt ; le travail (mesure, réponse, envoi) est fait par la boucle.

À la fin de chaque tour, l'instruction `__WFI()` (*Wait For Interrupt*) endort le processeur jusqu'à la prochaine interruption ; le SysTick le réveille au plus tard une milliseconde après. La boucle tourne ainsi au moins 1000 fois par seconde, et la consommation baisse quand il n'y a rien à faire (réglage `APP_SLEEP_WHEN_IDLE` de `app_config.h`).

> **La seule exception**
> `app_init()` attend volontairement : environ 150 ms pour le démarrage du module Bluetooth et quelques centaines de millisecondes pour les éclats de la LED. C'est sans conséquence : rien d'autre ne tourne encore.

4.4 Interruptions et données partagées

Une interruption peut survenir entre deux instructions de la boucle. Les variables qu'elle partage avec la boucle demandent donc des précautions :

Interruption	Fonction appelée	Ce qu'elle fait	Lu par la boucle avec
EXTI13 (bouton PC13)	`HAL_GPIO_EXTI_Callback()` → `button_on_interrupt()`	drapeau `s_edge = true`	`button_poll_pressed()`
TIM7 (1 s)	`HAL_TIM_PeriodElapsedCallback()` → `timer_1s_on_interrupt()`	compteur `s_ticks++`	`timer_1s_take_tick()`
USB (OTG_FS)	`CDC_Receive_FS()` → `usb_console_on_receive()`	note l'adresse et la taille du paquet reçu	`usb_console_read_line()`
EXTI12 (IRQ du module)	`hci_tl_lowlevel_isr()` (bibliothèque)	lit l'évènement du module par SPI	`hci_user_evt_proc()`
SysTick (1 ms)	`HAL_IncTick()`	compteur de millisecondes	`HAL_GetTick()`

- **`volatile`** : une variable modifiée par une interruption est déclarée `volatile` (exemple : `static volatile bool s_edge;` dans `button.c`). Sans ce mot, le compilateur pourrait garder sa valeur dans un registre et ne jamais voir le changement.
- **Section critique** : quand la boucle et une interruption modifient la même structure en plusieurs étapes, la boucle masque brièvement les interruptions (`irq_lock()` / `irq_unlock()` dans `usb_console.c`, qui sauvegardent et rétablissent le registre PRIMASK). Exemple : l'ajout d'un message à la file d'envoi USB, que `CDC_Transmit_FS()` peut aussi remplir.
- **Un seul écrivain, un seul lecteur** : chaque file (fifo) n'est remplie que d'un côté et vidée que de l'autre.

4.5 Qui appelle qui

La figure 7 compte, pour chaque fichier (en ligne), les appels aux fonctions de chaque module (en colonne). On y lit l'architecture : les fichiers générés n'appellent que les crochets ; `app.c` orchestre ; `commands.c` utilise les sorties, l'accéléromètre, la minuterie, les deux liaisons et `ota_switch` (commande `#ota`) ; `ota_switch.c` s'appuie sur les deux liaisons et sur `obcp_ota_request.h` ; `usb_console.c` et `ble_uart.c` s'appuient sur les outils `line_builder` et `fifo` ; les modules du bas n'appellent personne.

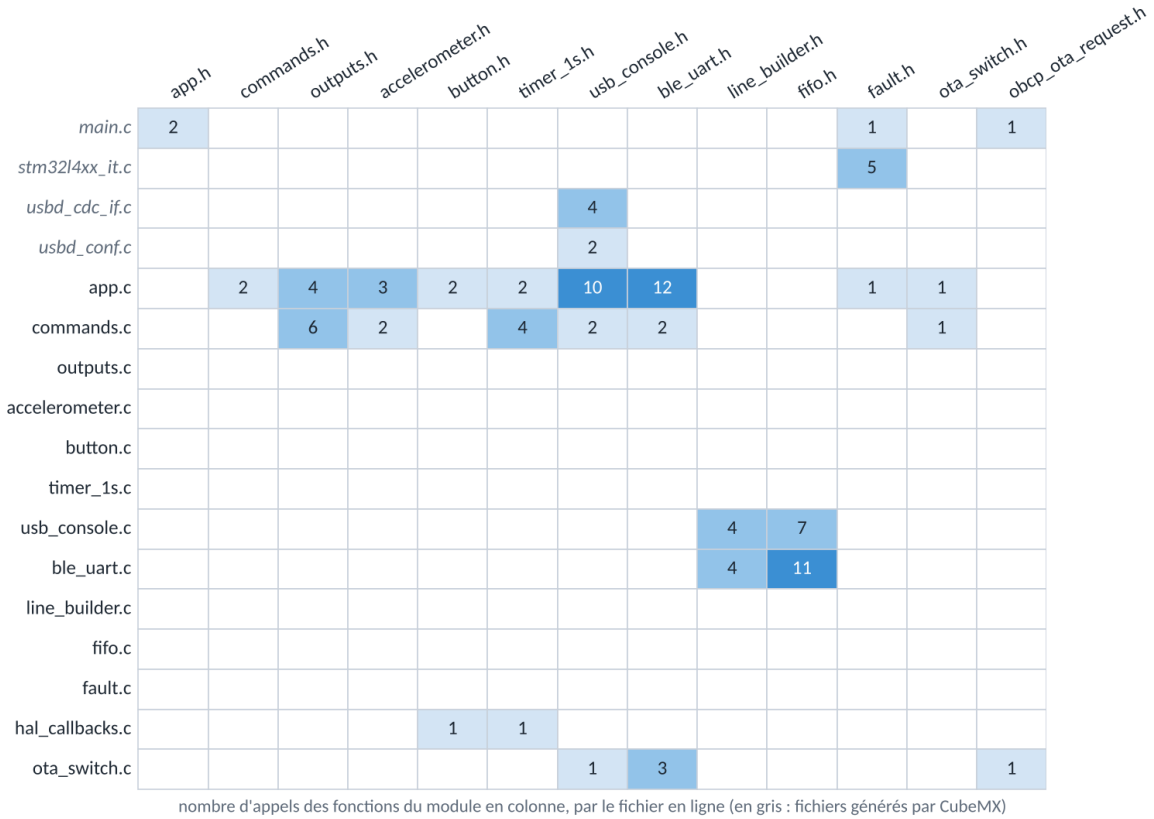


Figure 7 – *nombre d'appels entre les fichiers du programme (lignes) et les modules de l'application (colonnes)*

5. Fonctionnement détaillé des modules

Ce chapitre suit l'ordre de lecture conseillé : chaque section explique le principe d'un module, puis les points délicats. La liste complète des fonctions, avec leurs paramètres, est au chapitre 6. Chaque fichier commence aussi par un commentaire qui résume son fonctionnement.

5.1 app.c : démarrage et boucle principale

`app.c` est le chef d'orchestre. Il ne pilote aucun matériel lui-même : il appelle les autres modules dans le bon ordre.

Démarrage : app_init()

`main()` (généré) initialise la HAL, l'horloge et les périphériques (`MX_GPIO_Init()`, `MX_SPI3_Init()`, `MX_TIM1_Init()`… `MX_USB_DEVICE_Init()`), puis appelle `app_init()` :

Étape	Fonction	Effet
1	`fault_init()`	active les gestionnaires de faute détaillés (§ 5.11)
2	`outputs_init()`	démarre les quatre PWM, tout éteint
3	`accel_init(&hspi3)`	détecte l'accéléromètre (LIS3DH ou LIS3DSH) et le configure
4	`button_init()`	prépare l'anti-rebond du bouton
5	`ble_uart_init()`	démarre le module Bluetooth : adresse, nom, service UART (environ 150 ms)
6	`signal_startup()`	éclats de la LED : 1 vert si tout va bien, sinon 2 ou 3 rouges (§ 3.7)
7	`HAL_DBGMCU_EnableDBGSleepMode()`	version Debug seulement : garde la sonde ST-LINK connectée pendant la veille `__WFI()`

Boucle : app_loop()

Chaque tour enchaîne les douze étapes de la figure 6. Quelques détails :

- les commandes reçues sont lues avec `while (… read_line(…) != LINE_NONE)` : toutes les lignes arrivées sont traitées dans le même tour ;
- une **déconnexion** du téléphone est notée à l'étape 2, mais traitée à l'étape 5, après les commandes reçues juste avant elle : les sorties finissent bien éteintes ;
- l'étape 9 est la place réservée à votre code (§ 7.1) ;
- l'étape 11, `ota_switch_process()`, ne fait rien en temps normal ; après une commande `#ota`, elle conduit le passage au chargeur Bluetooth (§ 5.12) ;
- les messages spontanés (`Push button pressed`, `BLE disconnected :p`…) sont envoyés par de petites fonctions `on_ble_connected()`, `on_ble_disconnected()`, `send_measurements()`.

5.2 commands.c : les commandes

`commands.c` fait deux choses bien séparées :

- `commands_parse()` lit une ligne et dit de quelle commande il s'agit, sans rien toucher. C'est une fonction « pure » (même entrée, même résultat, aucun effet) : les tests sur PC l'essaient sur des centaines de cas ;
- `commands_execute()` appelle `commands_parse()`, agit (LED, sortie, capteur, minuterie) et envoie la réponse.

Analyse d'une ligne

1. les espaces et tabulations du début et de la fin sont retirés ; une ligne vide donne `CMD_EMPTY` (pas de réponse) ;

2. les commandes écrites en entier sont comparées **exactement** : `start timer`, `stop timer`, `#rst`, `#ota`, `cls`, `?` ;
3. les commandes d'une lettre sans valeur : `a`, `A`, `t`, `T` ;
4. une lettre suivie d'une valeur : `r`, `v`, `b`, `m` (majuscule ou minuscule). La fonction `parse_value()` accepte des espaces puis **uniquement des chiffres décimaux**, et s'arrête dès que le nombre dépasse 65535 (pas de débordement possible).

Ligne reçue	Résultat	Réponse
`r500`, `R500`, `r 500`, `r500`	`CMD_RED`, valeur 500	`Red = 500`
`r010`	`CMD_RED`, valeur 10 (décimal)	`Red = 10`
`r`, `r-1`, `r+5`, `r70000`, `r50x`, `r5.5`	`CMD_BAD_VALUE`	`False value (0-65535)`
`red`, `blue`, `hello`, `start timerX`	`CMD_UNKNOWN`	`False command`
(vide),	`CMD_EMPTY`	aucune

Réponse et recopie

La réponse est préparée dans un tableau local `reply` avec `snprintf()` (qui ne dépasse jamais la taille du tableau), puis confiée à `send_reply()` : envoi à l'interface d'origine, puis **recopie** sur l'autre si elle est active, comme en 2024. Les réponses vides (`start timer`, `#rst`…) ne sont pas envoyées. Deux exceptions : l'aide part seulement vers l'interface qui la demande, et `cls` n'a de sens que dans un terminal (par Bluetooth : `False command`).

`#ota` répond `OTA loader...` comme une commande ordinaire, puis demande le passage au chargeur Bluetooth (`ota_switch_request()`) : le redémarrage n'a lieu qu'une fois la réponse partie (§ 5.12).

> **Pourquoi le texte est-il copié ?**
> `usb_console_print()` et `ble_uart_print()` **copient** le texte dans leur file d'envoi avant de rendre la main. On peut donc leur passer un tableau local comme `reply`, qui disparaît à la fin de `commands_execute()`. En 2024, `CDC_Transmit_FS()` gardait seulement l'adresse du texte : l'USB pouvait envoyer un tableau déjà réutilisé.

5.3 line_builder.c : des caractères aux lignes

Les caractères arrivent un par un (frappe au clavier) ou par paquets (programme, téléphone). `line_builder_put()` les range dans une ligne de 64 caractères au plus, et rend `LINE_READY` quand une fin de ligne termine une ligne non vide :

Caractère reçu	Effet
`\r` (CR, 13) ou `\n` (LF, 10)	termine la ligne ; une ligne vide est ignorée, ce qui accepte aussi CR+LF (le LF arrive sur une ligne vide)
Backspace (8) ou Delete (127)	efface le dernier caractère
caractère nul (0)	ignoré
65e caractère et suivants	la ligne est marquée trop longue ; à la fin de ligne, elle est rejetée entière (`LINE_TOO_LONG`, réponse `False command`)

`line_builder_end()` termine la ligne en cours sans caractère de fin : c'est ainsi qu'un paquet USB de plusieurs caractères ou une écriture Bluetooth compte comme une commande complète.

5.4 fifo.c : le tampon circulaire

Un **tampon circulaire** (FIFO, « premier entré, premier sorti ») garde des octets en attente : le message est écrit d'un côté et lu de l'autre, plus tard, à son rythme. `fifo.c` utilise un tableau de taille fixe et deux compteurs (figure 8) :

- `head` : nombre total d'octets écrits depuis le début ; `tail` : nombre total d'octets retirés ;
- nombre d'octets présents = `head - tail` ; la case qui correspond à un compteur est `compteur & (taille - 1)` (un modulo rapide, d'où une taille puissance de 2) ;
- les compteurs sont des entiers 16 bits qui repassent à 0 après 65535 : la soustraction reste juste malgré ce « débordement ».

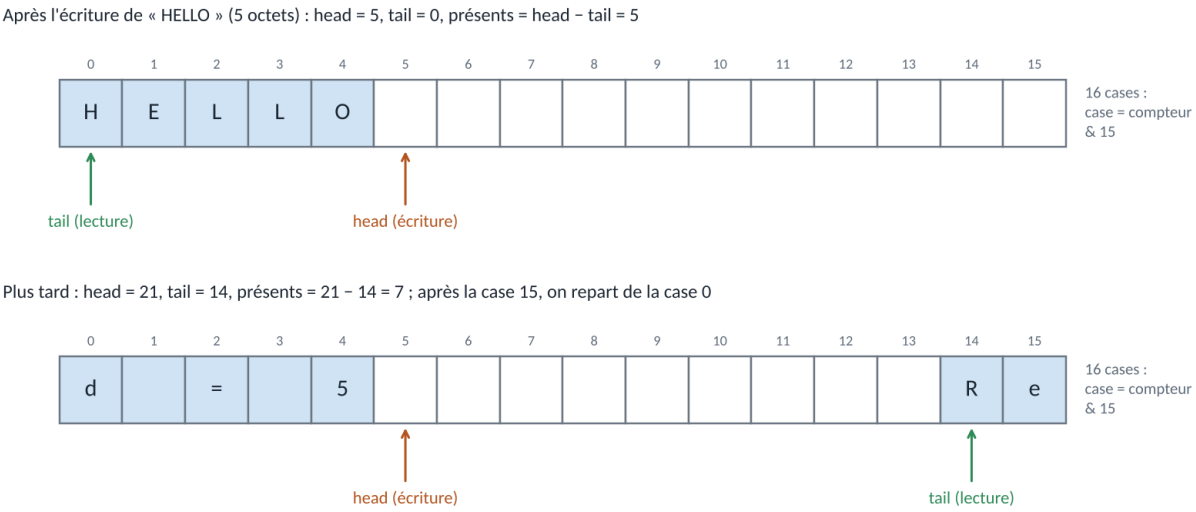


Figure 8 – *tampon circulaire de 16 cases : écriture, lecture, passage de la fin au début du tableau*

`fifo_write()` écrit **tout ou rien** : s'il manque de la place, le message entier est refusé, jamais coupé en deux. `fifo_peek()` lit sans retirer et `fifo_drop()` retire : on n'enlève les octets qu'une fois sûr qu'ils sont partis.

5.5 usb_console.c : le port COM virtuel

La bibliothèque USB de ST implémente la classe **CDC** (*Communication Device Class*) : le PC voit la carte comme un port série. Le code généré `usbd_cdc_if.c` reçoit ses évènements et les passe à `usb_console.c` par les crochets du § 4.2.

Réception, avec contrôle de flux

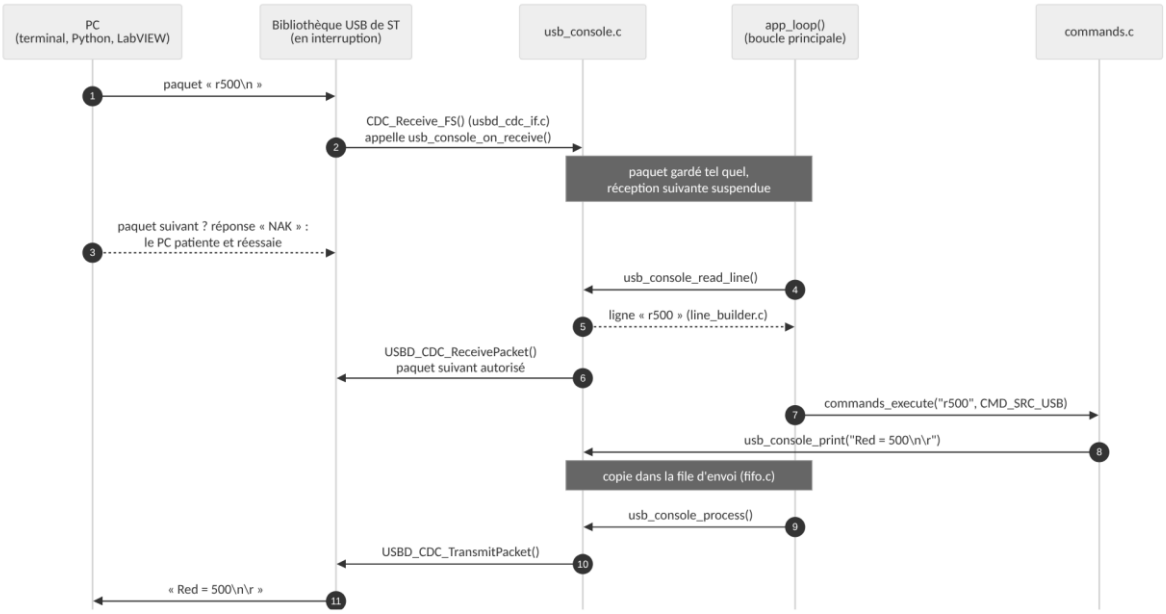


Figure 9 – *trajet d'une commande reçue du PC, jusqu'à la réponse*

Quand un paquet arrive (64 octets au plus), `CDC_Receive_FS()` est appelée en interruption. Elle note seulement l'adresse et la taille du paquet (`usb_console_on_receive()`). **La réception du paquet suivant n'est autorisée qu'une fois ce paquet lu** par la boucle (`USBD_CDC_ReceivePacket()` dans `finish_packet()`). En attendant, le contrôleur USB répond « NAK » (pas prêt) au PC, qui réessaie : c'est le **contrôle de flux** de l'USB, rien n'est perdu ni écrasé. La version 2024 autorisait le paquet suivant tout de suite : il pouvait remplacer la commande en cours de lecture.

Fin de commande : frappe ou programme ?

La règle de 2024 est conservée, pour que les terminaux et les programmes existants marchent tous :

Ce qui arrive	Interprétation	Exemple
un paquet d'**un seul caractère**	frappe au clavier : la commande se termine par Entrée	PuTTY : `r`, `5`, `0`, `0`, Entrée
un paquet de **plusieurs caractères** qui commence une ligne	envoi par un programme : la fin du paquet termine la commande, même sans fin de ligne	`ser.write(b"r500")`, option « Aucun » de `Terminal_port_com.py`
un paquet qui contient des fins de ligne	chaque fin de ligne termine une commande	`r500\r\nv200\r\n` = deux commandes

Émission

`usb_console_write()` (et `print`, `printf`) copie le texte dans une file de 1 ko et rend la main. `usb_console_process()`, appelée à chaque tour de boucle, confie au pilote jusqu'à 256 octets dès que le transfert précédent est fini (`TxState == 0`) ; le pilote les découpe en paquets de 64 octets. Si le PC ne lit plus, la file se remplit et les nouveaux messages sont refusés entiers (la fonction rend `false`) ; le programme, lui, n'est jamais bloqué.

`usb_console_is_ready()` indique si la carte est reconnue par le PC et si le bus n'est pas en veille (câble débranché). Elle ne dit pas si un terminal a ouvert le port : un message envoyé sans terminal ouvert attend dans la file. Après une déconnexion et une reconnexion, un compteur de « session » évite de réarmer la réception deux fois et fait oublier la ligne commencée avant.

`usb_console_tx_pending()` indique s'il reste des octets à envoyer (dans la file, ou en cours de transfert) : la commande `#ota` s'en sert pour attendre que sa réponse soit partie (§ 5.12).

5.6 ble_uart.c : la liaison Bluetooth

Le module BlueNRG-M0 contient toute la pile Bluetooth Low Energy. Le STM32 lui envoie des **commandes ACI** (*Application Command Interface*) par le bus SPI2, et le module signale ses **évènements** (connexion, écriture du téléphone…) en mettant sa broche IRQ (PB12) à 1. La bibliothèque BlueNRG-MS de ST fait ce dialogue ; `ble_uart.c` l'utilise. Toutes ses fonctions s'exécutent dans la boucle principale, jamais en interruption.

Démarrage : ble_uart_init()

Chaque étape est vérifiée ; en cas d'échec, `ble_uart_init()` rend un code `BLE_UART_ERR_…` qui dit laquelle, et la LED fait 3 éclats rouges au démarrage :

Étape	Appels	Rôle	Échec : BLE_UART…
1	`hci_init(ble_event_handler, NULL)`	prépare les files de paquets, reset du module, interruption IRQ	—
2	`getBlueNRGVersion()`	le module répond-il ?	`…_ERR_NO_MODULE`
3	`hci_reset()`, `HAL_Delay(100)`	redémarrage logiciel du module	—
4	`choose_address()`, `aci_hal_write_config_data()`	adresse de la carte	`…_ERR_ADDRESS`

Étape	Appels	Rôle	Échec : BLE_UART…
5	`aci_gatt_init()`, `aci_gap_init_IDB05A1()`, `aci_gatt_update_char_value()`	couches GATT et GAP, rôle « périphérique », nom OBCP-CROC	`…_ERR_GATT_INIT`, `…_ERR_GAP_INIT`, `…_ERR_DEVICE_NAME`
6	`aci_gap_set_auth_requirement()`	sécurité : code fixe 123456 si le téléphone demande l'appairage	`…_ERR_SECURITY`
7	`add_uart_service()`, `aci_hal_set_tx_power_level(1, 4)`	service UART (TX, RX) ; puissance d'émission	`…_ERR_SERVICE`, `…_ERR_TX_POWER`
8	`hci_user_evt_proc()`	évènements du redémarrage lus ; la publicité démarrera au premier `ble_uart_process()`	—

Adresse : comme en 2024, la carte utilise l'adresse « statique aléatoire » que le module a tirée au sort et gardée dans sa mémoire (`CONFIG_DATA_RANDOM_ADDRESS`), écrite comme adresse « publique ». Sur un module neuf, cette adresse n'existe qu'après le premier démarrage complet (note DT0070 de ST) : le programme la vérifie et, si elle manque, fabrique une adresse de secours à partir de l'identifiant unique du STM32 (hachage FNV-1a), différente d'une carte à l'autre.

Publicité, connexion, déconnexion

La carte « fait de la publicité » (*advertising*) : elle émet régulièrement un petit paquet avec son nom, que le téléphone voit lors d'une recherche. Le module traite ensuite la connexion tout seul et envoie des évènements que `ble_event_handler()` reçoit :

Évènement du module	Action
`EVT_LE_CONN_COMPLETE`, statut 0	connecté : l'évènement `BLE_EVENT_CONNECTED` est rangé pour `app_loop()`
`EVT_LE_CONN_COMPLETE`, statut ≠ 0	connexion échouée : la publicité est relancée (en 2024, la carte se croyait connectée et devenait introuvable)
`EVT_DISCONN_COMPLETE`	déconnecté : file d'envoi vidée, `BLE_EVENT_DISCONNECTED` rangé, publicité relancée
`EVT_BLUE_GATT_ATTRIBUTE_MODIFIED` sur RX	le téléphone a écrit une commande : texte copié, terminé par `'\0'`, rangé dans la file des messages (8 au plus)
`EVT_BLUE_GATT_ATTRIBUTE_MODIFIED` sur le descripteur de TX	le téléphone s'abonne aux notifications (ou se désabonne)
`EVT_BLUE_GATT_TX_POOL_AVAILABLE`	le module a de nouveau de la place : l'envoi reprend
`EVT_BLUE_HAL_INITIALIZED` (cause : chien de garde, plantage…)	le module a redémarré seul : tout est réinitialisé par la boucle

Si le module refuse la publicité (cela arrive juste après une déconnexion), `ble_uart_process()` réessaie toutes les 200 ms.

`ble_uart_disconnect()` ferme la connexion à l'initiative de la carte (commande `aci_gap_terminate()`, raison 0x13, « fermée par l'utilisateur ») ; le module le confirme par l'évènement `EVT_DISCONN_COMPLETE`, traité comme une déconnexion du téléphone. `ble_uart_tx_pending()` dit s'il reste des messages à envoyer. Les deux servent à la commande `#ota` (§ 5.12).

Réception d'une commande

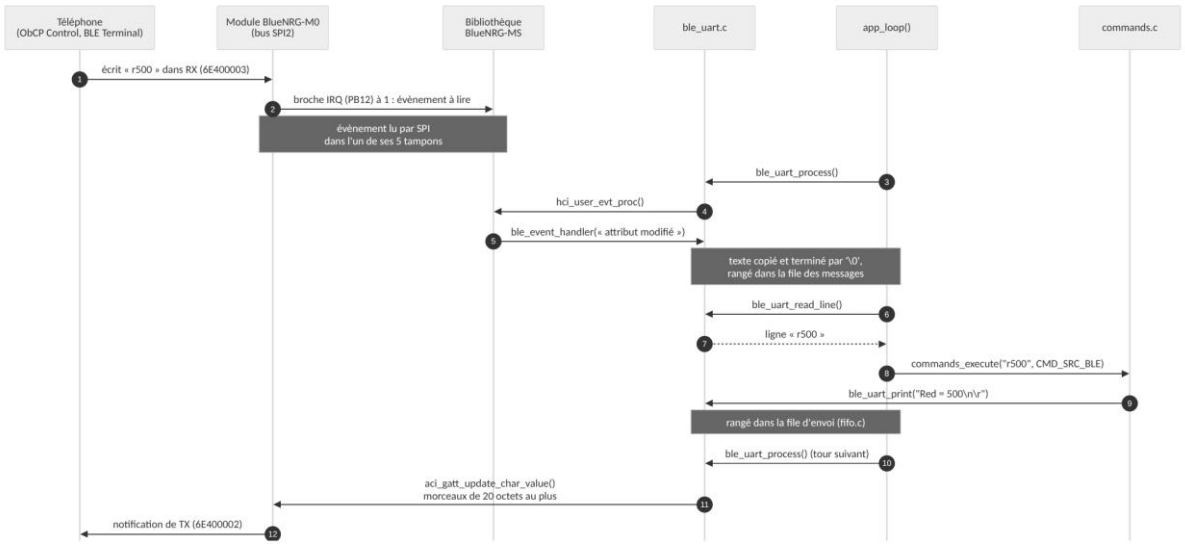


Figure 10 – *trajet d'une commande écrite par le téléphone, jusqu'à la notification de la réponse*

Le point clé est la **copie terminée par `'\0'`**. Le texte écrit par le téléphone se trouve au milieu d'un des 5 tampons de la bibliothèque, réutilisés à tour de rôle et jamais effacés. La version 2024 passait directement ce texte à `sscanf()`, qui lisait jusqu'au premier `'\0'` : derrière « r5 » se trouvaient encore les octets d'une commande précédente, et « r5 » devenait « r55535 ». `ble_uart_read_line()` découpe ensuite chaque message en lignes, la fin de l'écriture valant fin de ligne.

Envoi des notifications

`ble_uart_print()` range le message dans une file de 1 ko, précédé de sa longueur sur 2 octets. À chaque tour, `send_pending_notifications()` envoie au plus 4 morceaux de 20 octets avec `aci_gatt_update_char_value()`, en commençant chaque message dans une nouvelle notification. Si le module répond qu'il n'a plus de tampon libre (`BLE_STATUS_INSUFFICIENT_RESOURCES`, le téléphone ne suit pas), l'envoi attend l'évènement `TX_POOL_AVAILABLE` (ou 20 ms) et reprend au même morceau : **rien n'est perdu**, et la boucle n'est jamais bloquée (la version 2024 attendait jusqu'à 10 ms par morceau, puis abandonnait).

> **Une sécurité de plus**
> Si les 5 tampons de la bibliothèque sont pleins quand le module signale un évènement, l'interruption ne le lit pas ; or la broche IRQ, restée à 1, ne produira plus de front montant. `read_pending_packets()` regarde donc PB12 à chaque tour et relance la lecture si elle est restée à 1.

5.7 outputs.c : LED et sortie de puissance

Une LED alimentée en **PWM** est allumée puis éteinte des centaines de fois par seconde ; l'œil voit une luminosité moyenne proportionnelle au **rapport cyclique** (fraction du temps à l'état allumé). `outputs.c` associe chaque sortie à une voie de minuterie :

Sortie	Broche	Minuterie, voie	Commande
`OUTPUT_RED`	PC6	TIM3, voie 1	`rn`
`OUTPUT_GREEN`	PC8	TIM3, voie 3	`vn`
`OUTPUT_BLUE`	PC9	TIM3, voie 4	`bn`
`OUTPUT_MOTOR`	PB1	TIM1, voie 3 complémentaire (CH3N)	`mn`

`outputs_set(out, n)` écrit *n* dans le registre de comparaison CCR (`__HAL_TIM_SET_COMPARE`) ; le rapport cyclique vaut *n* / 65536. Le registre est « préchargé » (*preload*) : la nouvelle valeur ne prend effet qu'au début de la période suivante, ce qui évite une impulsion de longueur fausse.

5.8 accelerometer.c : l'accéléromètre

Le capteur dialogue en **SPI** : la carte (maître) met sa broche de sélection CS à 0, envoie un premier octet qui dit quoi faire, puis échange les données ; CS repasse à 1 à la fin. En SPI, chaque octet envoyé fait revenir un octet : pour lire, on envoie des octets sans importance pendant que le capteur renvoie ses registres (figure 11).

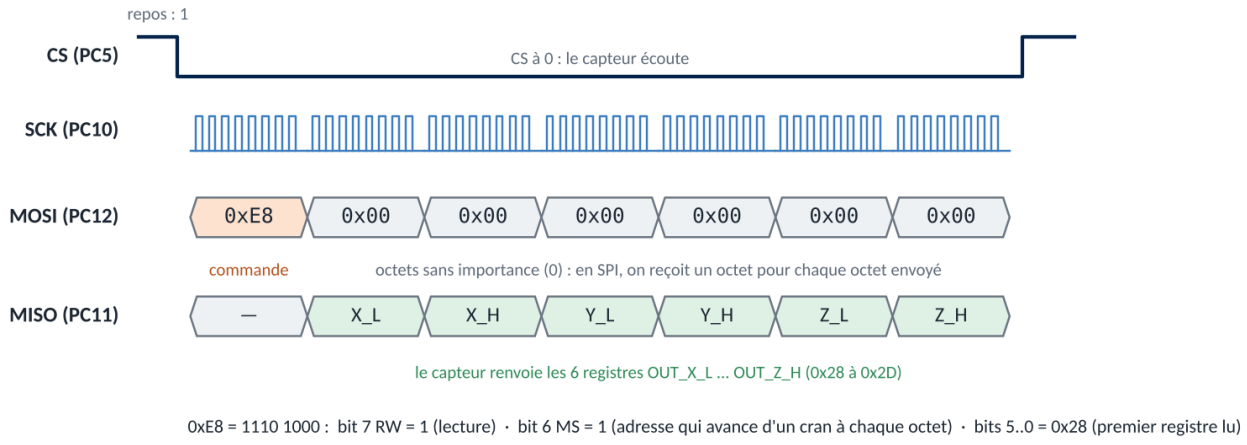


Figure 11 – *lecture des trois axes en un seul échange SPI (LIS3DH)*

Détection et configuration

`accel_init()` lit le registre d'identité `WHO_AM_I` (adresse 0x0F) : 0x33 pour un LIS3DH, 0x3F pour un LIS3DSH, autre chose si aucun capteur ne répond. Il écrit ensuite la configuration et **relit chaque registre** pour vérifier que le capteur l'a bien prise :

Circuit	Registre	Valeur	Signification
LIS3DH	CTRL_REG1 (0x20)	0x27	10 mesures par seconde, axes X, Y, Z actifs
LIS3DH	CTRL_REG4 (0x23)	0x88	mise à jour par bloc (BDU), ±2 g, haute résolution (12 bits)
LIS3DH	TEMP_CFG_REG (0x1F)	0xC0	convertisseur et capteur de température actifs
LIS3DSH	CTRL_REG4 (0x20)	0x3F	12,5 mesures par seconde, BDU, axes X, Y, Z
LIS3DSH	CTRL_REG5 (0x24)	0x00	±2 g, filtre 800 Hz
LIS3DSH	CTRL_REG6 (0x25)	0x10	avance automatique de l'adresse (lecture des 6 octets d'un coup)

Conversion en g

Les six registres `OUT_X_L` à `OUT_Z_H` (0x28 à 0x2D) donnent trois entiers signés de 16 bits. Avec la mise à jour par bloc et la lecture des 6 octets en un seul échange, les trois axes viennent toujours de la même mesure. La conversion :

```
mg = brut x sensibilité        (LIS3DH : 0,0625 mg ; LIS3DSH : 0,06 mg par unité, à ±2 g)
g  = (mg - décalage) x échelle / 1000
```

Le LIS3DH en haute résolution donne 1 mg par pas sur 12 bits, valeur alignée à gauche dans 16 bits (multipliée par 16) : d'où 1/16 = 0,0625 mg par unité. La version 2024 appliquait 0,06 (la valeur du LIS3DSH) : ses accélérations étaient 4 % trop faibles (Z = 0,96 au lieu de 1,00 carte à plat).

Étalonnage

`accel_set_calibration(axe, min_mg, max_mg)` corrige un axe à partir de deux mesures : axe vers le bas (−1 g attendu, `min_mg`) puis vers le haut (+1 g attendu, `max_mg`). Le décalage vaut (max + min) / 2 et l'échelle 2000 / (max − min) : la mesure `max_mg` devient +1000 mg et `min_mg` −1000 mg. Exemple, pour l'axe Z mesuré à −980 mg carte retournée et +1020 mg carte à plat (commande `a`) :

```
accel_set_calibration(ACCEL_AXIS_Z, -980.0f, 1020.0f);   /* dans app_init() */
```

Température

Le capteur mesure les variations de sa température, pas sa valeur absolue. La formule « valeur lue + 25 °C », reprise de 2024, donne un ordre de grandeur à quelques degrés près : utile pour voir une variation (doigt posé sur le capteur), pas comme thermomètre.

5.9 button.c : le bouton et ses rebonds

Un contact mécanique **rebondit** : en s'enfonçant, il s'ouvre et se ferme plusieurs fois pendant quelques millisecondes. Sans précaution, un seul appui compterait pour plusieurs. Le bouton SW1 relie PC13 à la masse (résistance de rappel R1 vers 3,3 V) : relâché, PC13 = 1 ; enfoncé, PC13 = 0.

L'interruption EXTI13 (front descendant) ne fait que lever un drapeau. La boucle appelle `button_poll_pressed()`, une **machine à états** (figure 12) : un appui n'est compté que si le bouton reste enfoncé 30 ms, et le suivant n'est accepté qu'après 30 ms de relâchement. Aucune attente : chaque appel regarde l'état et l'heure, et rend la main.

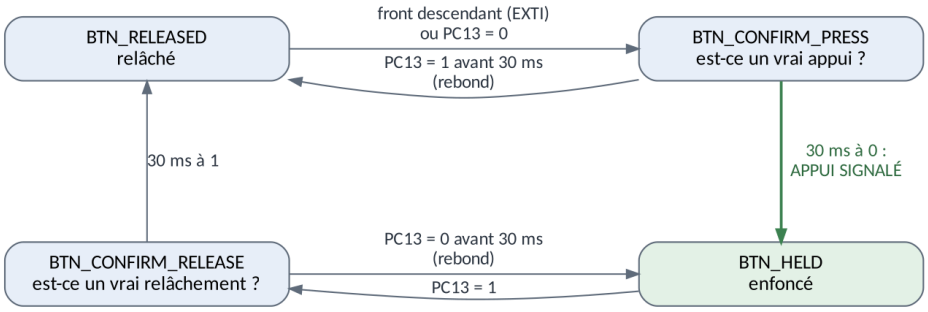


Figure 12 – *machine à états de l'anti-rebond (BUTTON_DEBOUNCE_MS = 30 ms)*

La version 2024 attendait 50 ms (`HAL_Delay`), puis le relâchement du bouton dans une boucle `while`, puis encore 50 ms : tant que le bouton restait enfoncé, tout le programme était arrêté.

5.10 timer_1s.c : la minuterie d'une seconde

TIM7 produit une interruption par seconde (§ 2.2). L'interruption appelle `timer_1s_on_interrupt()`, qui ajoute 1 à un compteur `volatile`. La boucle appelle `timer_1s_take_tick()`, qui rend `true` une fois par seconde écoulée ; la mesure et l'envoi, qui prennent du temps, sont faits dans la boucle (`send_measurements()` de `app.c`).

`timer_1s_start()` remet le compteur de TIM7 à zéro et efface son drapeau d'interruption : la première mesure arrive une seconde pleine après `start timer`. Une seconde commande `start timer` ne change rien ; `stop timer`, `#rst` et la déconnexion du téléphone arrêtent la minuterie.

5.11 fault.c et hal_callbacks.c

fault.c signale les erreurs graves sur une carte sans débogueur. `fault_signal(cause)` masque les interruptions, remet LED et sortie J13 en sorties ordinaires à 0 (sécurité : le moteur s'arrête), puis fait clignoter la LED rouge sans fin, le nombre d'éclats donnant la cause (§ 3.7). Comme les interruptions sont masquées, `HAL_Delay()` ne marche plus (elle compte les interruptions du SysTick) : `wait_ms()` compte directement les passages à 1 du drapeau `COUNTFLAG` du SysTick, qui continue de tourner.

`fault_init()`, appelée au début de `app_init()`, active les gestionnaires **MemManage**, **BusFault** et **UsageFault** (registre SHCSR) et le piège de la **division entière par zéro** (bit DIV_0_TRP du registre CCR) : sans elle, le Cortex-M4 range toutes ces fautes dans HardFault, et une division par zéro donne 0 sans rien dire.

hal_callbacks.c contient les deux fonctions de rappel que la HAL appelle depuis les interruptions : `HAL_GPIO_EXTI_Callback()` (vérifie qu'il s'agit de `USER_BUTTON_Pin` et prévient `button.c`) et `HAL_TIM_PeriodElapsedCallback()` (vérifie qu'il s'agit de TIM7 et prévient `timer_1s.c`). La HAL en fournit des versions vides marquées `__weak` : celles de l'application les remplacent à l'édition de liens. Une fonction de rappel ne doit exister qu'une fois dans tout le programme (sinon : erreur « multiple definition »).

5.12 ota_switch.c : le passage au chargeur Bluetooth

La commande `#ota` fait redémarrer la carte dans le chargeur Bluetooth (§ 3.1). `commands_execute()` éteint les sorties, arrête la minuterie, prépare la réponse `OTA loader...` et appelle `ota_switch_request()` ; le passage lui-même est conduit par `ota_switch_process()`, à l'étape 11 de la boucle.

Pourquoi pas un redémarrage immédiat ? La réponse n'est encore que dans les files d'envoi : un reset la couperait en route. Et un téléphone connecté ne s'apercevrait de la coupure qu'au bout du délai de supervision de la liaison, plusieurs secondes plus tard. Le passage se fait donc en trois étapes (figure 13), sans jamais attendre sur place :

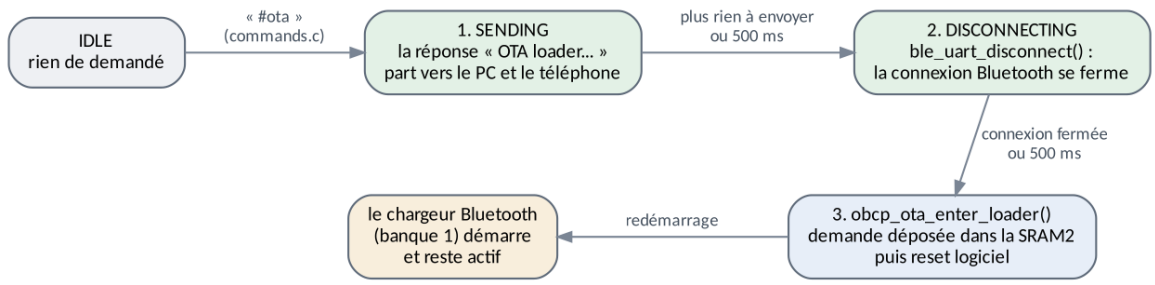


Figure 13 – *la commande #ota : passage au chargeur Bluetooth en trois étapes (ota_switch.c)*

1. **envoi** (`OTA_STEP_SENDING`) : attendre que la réponse soit partie vers le PC et le téléphone (`usb_console_tx_pending()`, `ble_uart_tx_pending()`) ;
2. **déconnexion** (`OTA_STEP_DISCONNECTING`) : fermer la connexion Bluetooth (`ble_uart_disconnect()`), attendre que le module le confirme, puis que le message `BLE disconnected :p` soit parti vers le PC ;
3. **redémarrage** : `obcp_ota_enter_loader()` dépose une demande dans la « boîte aux lettres » de la SRAM2 (deux mots de 32 bits à `0x10007FF0` : un mot « magique » et son complément), puis provoque un reset logiciel. Au redémarrage, le chargeur trouve la demande, l'efface et reste actif.

Chaque attente est limitée à 500 ms (`OTA_REPLY_TIMEOUT_MS` et `OTA_DISCONNECT_TIMEOUT_MS`, § 6.13) : si le PC ne lit plus le port COM, ou si le module ne confirme pas la déconnexion, la carte redémarre quand même, au bout d'une seconde au plus. Une seconde demande pendant le passage est sans effet.

> Une machine à états
>
> La variable `s_step` dit à quelle étape on en est ; chaque appel de `ota_switch_process()` regarde si l'étape en cours est terminée et passe à la suivante. C'est la façon habituelle d'écrire une suite d'actions qui prend du temps sans bloquer la boucle : pendant l'attente, l'USB et le Bluetooth continuent d'être servis… et envoient justement la réponse. L'anti-rebond du bouton fonctionne de la même façon (§ 5.9).

`obcp_ota_request.h`, copie du fichier du projet ObCP_OTA_BLE, fournit aussi `obcp_ota_init_vectors()`, appelée au tout début de `main()` (section USER CODE 1). Le registre VTOR du processeur indique où se trouve la table des interruptions ; après un reset, il vaut 0, c'est-à-dire la table… du chargeur. Le chargeur règle VTOR avant de lancer le programme ; cet appel rend le programme autonome, par exemple lancé directement par le

débogueur. Sur le PC, les tests remplacent `obcp_ota_enter_loader()` par une fonction qui compte ses appels et revient (§ 8.3).

6. Référence des fonctions

Ce chapitre est extrait automatiquement des fichiers `.h` de l'application : chaque fonction y est décrite par un commentaire placé juste avant sa déclaration. La colonne de gauche donne la déclaration complète (type rendu, nom, paramètres).

6.1 app : démarrage et boucle principale

Fichiers `Core/Inc/app.h` (31 lignes) et `Core/Src/app.c` (299 lignes). Application ObCP : démarrage (app_init) et boucle principale (app_loop), appelés par main.c.

Fonction	Rôle
`void app_init(void)`	Démarre l'application : sorties PWM, accéléromètre, bouton, Bluetooth, puis signal de démarrage par la LED (voir app.c). À appeler une fois, après les MX_xxx_Init() de main.c.
`void app_loop(void)`	Un tour de la boucle principale. À appeler sans fin dans le while (1) de main.c.

6.2 commands : analyse et exécution des commandes

Fichiers `Core/Inc/commands.h` (111 lignes) et `Core/Src/commands.c` (355 lignes). Interpréteur des commandes reçues par le port COM virtuel (USB) ou par Bluetooth : analyse de la ligne, action, réponse.

Fonction	Rôle
`cmd_t commands_parse(const char *line)`	Analyse une ligne de commande, sans rien exécuter (fonction « pure », testée sur PC dans Tests/test_unit.c). `line` : ligne terminée par '\0', sans caractère de fin de ligne **Rend** : commande reconnue et sa valeur
`void commands_execute(const char *line, cmd_source_t src)`	Analyse et exécute une ligne, puis envoie la réponse. `line` : ligne terminée par '\0' `src` : interface d'où vient la ligne (pour savoir où répondre)
`void commands_reject(cmd_source_t src)`	Répond « False command » à une ligne rejetée avant analyse (ligne trop longue, voir line_builder.h). `src` : interface d'où vient la ligne
`const char *commands_help_text(void)`	Texte d'aide affiché par la commande « ? ».

Type	Description et valeurs
`cmd_source_t`	Interface par laquelle une commande est arrivée. `CMD_SRC_USB` : port COM virtuel (câble USB) `CMD_SRC_BLE` : Bluetooth (téléphone)

Type	Description et valeurs
`cmd_id_t`	Commandes reconnues par commands_parse(). `CMD_EMPTY` : ligne vide ou faite d'espaces : rien à faire `CMD_RED` : r<n> `CMD_GREEN` : v<n> `CMD_BLUE` : b<n> `CMD_MOTOR` : m<n> `CMD_ACCEL` : a `CMD_TEMPERATURE` : t `CMD_HELP` : ? `CMD_CLEAR_SCREEN` : cls `CMD_TIMER_START` : start timer `CMD_TIMER_STOP` : stop timer `CMD_RESET` : #rst `CMD_OTA` : #ota : redémarrage dans le chargeur Bluetooth `CMD_BAD_VALUE` : r, v, b ou m avec une valeur absente ou invalide `CMD_UNKNOWN` : commande inconnue
`cmd_t`	Résultat de l'analyse d'une ligne. `cmd_id_t id` : commande reconnue `uint16_t value` : valeur de r, v, b, m (0 à 65535)

6.3 outputs : LED et sortie moteur (PWM)

Fichiers `Core/Inc/outputs.h` (67 lignes) et `Core/Src/outputs.c` (75 lignes). Sorties PWM de l'ObCP : LED tricolore (rouge, verte, bleue) et sortie de puissance du bornier J13 (« moteur »).

Fonction	Rôle
`void outputs_init(void)`	Démarre les quatre PWM avec un rapport cyclique nul (tout éteint). À appeler une fois, après MX_TIM1_Init() et MX_TIM3_Init().
`void outputs_set(output_t out, uint16_t value)`	Règle une sortie. `out` : sortie (OUTPUT_RED...) `value` : 0 (éteint) à 65535 (pleine puissance)
`uint16_t outputs_get(output_t out)`	Dernière valeur réglée pour une sortie.
`void outputs_all_off(void)`	Éteint les quatre sorties (commande « #rst » et déconnexion BLE).

Type	Description et valeurs
`output_t`	Les quatre sorties PWM. `OUTPUT_RED` : LED rouge (TIM3, voie 1) `OUTPUT_GREEN` : LED verte (TIM3, voie 3) `OUTPUT_BLUE` : LED bleue (TIM3, voie 4) `OUTPUT_MOTOR` : sortie du bornier J13 (TIM1, voie 3 complémentaire) `OUTPUT_COUNT` : nombre de sorties (pas une sortie)

6.4 accelerometer : accéléromètre LIS3DH / LIS3DSH

Fichiers `Core/Inc/accelerometer.h` (122 lignes) et `Core/Src/accelerometer.c` (292 lignes). Accéléromètre 3 axes de l'ObCP (U4), relié au bus SPI3.

Fonction	Rôle
`accel_chip_t accel_init(SPI_HandleTypeDef *hspi)`	Détecte et configure le capteur. `hspi` : bus SPI du capteur (&hspi3, initialisé par MX_SPI3_Init) **Rend** : circuit trouvé, ACCEL_NONE si aucun ne répond correctement
`accel_chip_t accel_chip(void)`	Circuit détecté par accel_init().
`const char *accel_chip_name(void)`	Nom du circuit détecté : "LIS3DH", "LIS3DSH" ou "none".
`bool accel_read_raw(int16_t raw[3])`	Lit les valeurs brutes des trois axes (entiers signés 16 bits, tels que le capteur les fournit), lues en une seule fois pour que les trois axes viennent de la même mesure. `raw` : tableau de 3 cases : raw[0] = X, raw[1] = Y, raw[2] = Z **Rend** : true si la lecture a réussi
`bool accel_read(accel_g_t *g)`	Lit l'accélération sur les trois axes, en g, étalonnage compris. `g` : résultat **Rend** : true si la lecture a réussi, false si le capteur est absent ou ne répond pas
`bool accel_read_temperature(int *celsius)`	Lit la température du capteur, en °C (entier). `celsius` : résultat **Rend** : true si la lecture a réussi Le capteur mesure les variations de sa température, pas sa valeur absolue : la formule « valeur lue + 25 °C », reprise de la version 2024, donne un ordre de grandeur (à quelques degrés près). Utile pour voir une variation (doigt posé sur le capteur), pas comme thermomètre.

`void accel_set_calibration(accel_axis_t axis, float min_mg, float max_mg)`	Étalonnage simple d'un axe, à partir des valeurs mesurées (en mg) quand cet axe pointe vers le haut (+1 g attendu, valeur max_mg : carte à plat pour Z) puis vers le bas (-1 g attendu, valeur min_mg : carte retournée pour Z). Ensuite, chaque mesure est corrigée : (mesure - décalage) x échelle, avec décalage = (max_mg + min_mg) / 2 et échelle = 2000 / (max_mg - min_mg) : max_mg devient +1000 mg et min_mg devient -1000 mg.
	`axis` : axe à étalonner
	`min_mg` : mesure brute à -1 g, en mg
	`max_mg` : mesure brute à +1 g, en mg (doit être > min_mg)
	Exemple (valeurs relevées avec la commande « a ») :
	`accel_set_calibration(ACCEL_AXIS_X, -1000.0f, 980.0f);`
	`accel_set_calibration(ACCEL_AXIS_Y, -1020.0f, 1040.0f);`
	`accel_set_calibration(ACCEL_AXIS_Z, -920.0f, 1040.0f);`

Type	Description et valeurs
`accel_chip_t`	Circuit détecté au démarrage. `ACCEL_NONE` : aucun capteur ne répond (absent, mal soudé...) `ACCEL_LIS3DH` : LIS3DH, WHO_AM_I = 0x33 `ACCEL_LIS3DSH` : LIS3DSH, WHO_AM_I = 0x3F

`accel_axis_t`	Axe, pour accel_set_calibration().
	`ACCEL_AXIS_X`
	`ACCEL_AXIS_Y`
	`ACCEL_AXIS_Z`

`accel_g_t`	Accélération sur les trois axes, en g.
	`float x`
	`float y`
	`float z`

6.5 button : bouton utilisateur

Fichiers `Core/Inc/button.h` (52 lignes) et `Core/Src/button.c` (102 lignes). Bouton utilisateur SW1 (« User Push Button » du schéma), sur PC13.

Fonction	Rôle
`void button_init(void)`	Prépare la détection (à appeler une fois au démarrage).
`void button_on_interrupt(void)`	À appeler par l'interruption EXTI du bouton (HAL_GPIO_EXTI_Callback, dans hal_callbacks.c).
`bool button_poll_pressed(void)`	Boucle principale : rend true une fois par appui (anti-rebond fait).
`bool button_is_down(void)`	État instantané du bouton (true = enfoncé), sans anti-rebond.

6.6 timer_1s : minuterie d'une seconde

Fichiers `Core/Inc/timer_1s.h` (52 lignes) et `Core/Src/timer_1s.c` (71 lignes). Minuterie d'une seconde (TIM7) des commandes « start timer » et « stop timer » : chaque seconde, l'application envoie l'accélération et la température au téléphone.

Fonction	Rôle
`void timer_1s_start(void)`	Démarre la minuterie : premier « tic » une seconde plus tard.
`void timer_1s_stop(void)`	Arrête la minuterie (un tic en attente est oublié).
`bool timer_1s_is_running(void)`	Indique si la minuterie tourne.
`void timer_1s_on_interrupt(void)`	À appeler par l'interruption de TIM7 (HAL_TIM_PeriodElapsedCallback, dans hal_callbacks.c) : note qu'une seconde s'est écoulée.
`bool timer_1s_take_tick(void)`	Boucle principale : rend true une fois par seconde écoulée.

6.7 usb_console : port COM virtuel (USB)

Fichiers `Core/Inc/usb_console.h` (131 lignes) et `Core/Src/usb_console.c` (350 lignes). Port COM virtuel par le câble USB (classe USB « CDC ») : réception des lignes de commande, envoi des réponses vers le PC.

Fonction	Rôle
`line_status_t usb_console_read_line(char *line, size_t size)`	Rend la prochaine ligne de commande reçue du PC. `line` : destination (CMD_LINE_MAX + 1 octets conseillés) `size` : taille de line en octets **Rend** : LINE_READY : une ligne est dans line (terminée par '\0') ; LINE_TOO_LONG : une ligne trop longue a été reçue et rejetée ; LINE_NONE : plus rien à lire pour le moment
`bool usb_console_write(const uint8_t *data, uint16_t len)`	Confie des octets à envoyer au PC (copiés aussitôt). `data` : octets à envoyer `len` : nombre d'octets **Rend** : true si les octets sont dans le tampon ; false si l'USB n'est pas prêt (câble débranché) ou si le tampon est plein (le PC ne lit plus) : le message est alors abandonné, entier
`bool usb_console_print(const char *text)`	Envoie une chaîne de caractères (terminée par '\0') au PC. **Rend** : comme usb_console_write()
`bool usb_console_printf(const char *format, ...)`	Envoie un texte mis en forme comme avec printf (127 caractères au plus), par exemple usb_console_printf("x = %d\n\r", x). **Rend** : comme usb_console_write()
`void usb_console_process(void)`	À appeler dans la boucle principale : envoie au PC le contenu du tampon d'émission, morceau par morceau.
`bool usb_console_is_ready(void)`	Indique si l'USB est utilisable : la carte est reconnue (« énumérée ») par le PC et le bus n'est pas en veille. Ne dit pas si un terminal a ouvert le port.
`bool usb_console_take_link_up_event(void)`	Rend true une seule fois après chaque connexion (ou réveil) de l'USB : sert à envoyer « USB Connected » au téléphone.

Fonction	Rôle
`bool usb_console_tx_pending(void)`	Indique s'il reste des octets à envoyer au PC (dans le tampon, ou en cours de transfert). Toujours false si l'USB n'est pas prêt : ces octets ne partiraient jamais. Sert à la commande « #ota » (ota_switch.c) : on attend que la réponse soit partie.
`void usb_console_on_init(void)`	Le PC a configuré le port (fin de l'énumération) : CDC_Init_FS().
`void usb_console_on_deinit(void)`	Le port n'est plus configuré (débranchement, reset du bus) : CDC_DeInit_FS().
`void usb_console_on_receive(uint8_t *buf, uint32_t len)`	Un paquet de len octets est arrivé dans buf : CDC_Receive_FS().
`void usb_console_on_suspend(void)`	Bus USB en veille (câble débranché, PC endormi) : HAL_PCD_SuspendCallback().
`void usb_console_on_resume(void)`	Fin de veille du bus USB : HAL_PCD_ResumeCallback().

6.8 ble_uart : liaison Bluetooth

Fichiers `Core/Inc/ble_uart.h` (162 lignes) et `Core/Src/ble_uart.c` (822 lignes). Liaison Bluetooth Low Energy (BLE) « façon port série » avec un téléphone, par le module BlueNRG-M0 de l'ObCP (BLE1).

Fonction	Rôle
`ble_uart_status_t ble_uart_init(void)`	Démarre le module Bluetooth : adresse, nom, service UART, puissance d'émission. La publicité démarre ensuite d'elle-même dans ble_uart_process(). Durée : environ 150 ms. **Rend** : BLE_UART_OK, ou l'étape qui a échoué (le reste du programme fonctionne sans Bluetooth)
`void ble_uart_process(void)`	À appeler dans la boucle principale : traite les évènements du module, relance la publicité après une déconnexion et envoie les notifications en attente. Ne bloque pas longtemps (quelques ms au plus).
`bool ble_uart_is_ready(void)`	Indique si le module a été initialisé avec succès.
`bool ble_uart_is_connected(void)`	Indique si un téléphone est connecté.
`line_status_t ble_uart_read_line(char *line, size_t size)`	Rend la prochaine ligne de commande reçue du téléphone. Chaque écriture du téléphone compte comme une ligne complète, même sans fin de ligne ; si elle contient « \r » ou « \n », elle est découpée en plusieurs lignes. `line` : destination (CMD_LINE_MAX + 1 octets conseillés) `size` : taille de line **Rend** : LINE_READY (ligne dans line), LINE_TOO_LONG ou LINE_NONE
`bool ble_uart_send(const uint8_t *data, uint16_t len)`	Range un message à envoyer au téléphone (copié aussitôt). Il partira en notifications de 20 octets au plus. `data` : octets à envoyer `len` : nombre d'octets **Rend** : true si le message est en attente d'envoi ; false si aucun téléphone n'est connecté ou si le tampon est plein
`bool ble_uart_print(const char *text)`	Envoie une chaîne de caractères (terminée par '\0') au téléphone. **Rend** : comme ble_uart_send()
`bool ble_uart_printf(const char *format, ...)`	Envoie un texte mis en forme comme avec printf (127 caractères au plus) au téléphone. **Rend** : comme ble_uart_send()
`ble_uart_event_t ble_uart_get_event(uint8_t peer[6])`	Rend le prochain évènement de connexion ou de déconnexion. `peer` : si non NULL et BLE_EVENT_CONNECTED : adresse du téléphone (6 octets, octet de poids faible en premier) **Rend** : BLE_EVENT_NONE s'il n'y a rien de nouveau
`void ble_uart_get_address(uint8_t address[6])`	Adresse Bluetooth de la carte (6 octets, poids faible en premier).

Fonction	Rôle
`bool ble_uart_tx_pending(void)`	Indique s'il reste des messages à envoyer au téléphone (false si aucun téléphone n'est connecté). Sert à la commande « #ota » (ota_switch.c) : on attend que la réponse soit partie.
`void ble_uart_disconnect(void)`	Ferme la connexion avec le téléphone (sans effet s'il n'y en a pas). La fin de la connexion est confirmée un peu plus tard par le module : ble_uart_is_connected() devient false, et ble_uart_get_event() rend BLE_EVENT_DISCONNECTED.

Type	Description et valeurs
`ble_uart_status_t`	Résultat de ble_uart_init() : BLE_UART_OK ou l'étape qui a échoué. `BLE_UART_OK` : module prêt `BLE_UART_ERR_NO_MODULE` : le module ne répond pas (absent, SPI, reset...) `BLE_UART_ERR_ADDRESS` : adresse Bluetooth refusée `BLE_UART_ERR_GATT_INIT` : initialisation GATT refusée `BLE_UART_ERR_GAP_INIT` : initialisation GAP refusée `BLE_UART_ERR_DEVICE_NAME` : nom de l'appareil refusé `BLE_UART_ERR_SECURITY` : réglage de sécurité refusé `BLE_UART_ERR_SERVICE` : création du service UART refusée `BLE_UART_ERR_TX_POWER` : réglage de la puissance d'émission refusé
`ble_uart_event_t`	Évènement de connexion rendu par ble_uart_get_event(). `BLE_EVENT_NONE` : rien de nouveau `BLE_EVENT_CONNECTED` : un téléphone vient de se connecter `BLE_EVENT_DISCONNECTED` : le téléphone vient de se déconnecter

6.9 line_builder : assemblage des lignes de commande

Fichiers `Core/Inc/line_builder.h` (89 lignes) et `Core/Src/line_builder.c` (97 lignes). Assemblage des lignes de commande à partir des caractères reçus (USB ou Bluetooth).

Fonction	Rôle
`void line_builder_reset(line_builder_t *lb)`	Vide la ligne en cours.
`line_status_t line_builder_put(line_builder_t *lb, char c)`	Ajoute un caractère reçu à la ligne en cours. `lb` : ligne en cours `c` : caractère reçu **Rend** : LINE_READY quand c termine une ligne non vide (lire lb->text), LINE_TOO_LONG quand c termine une ligne trop longue, LINE_NONE sinon
`line_status_t line_builder_end(line_builder_t *lb)`	Termine la ligne en cours comme si une fin de ligne était reçue (fin d'un message reçu en un seul bloc). **Rend** : LINE_READY, LINE_TOO_LONG ou LINE_NONE (rien en cours)
`bool line_builder_is_empty(const line_builder_t *lb)`	Indique si aucun caractère n'attend dans la ligne en cours.

Type	Description et valeurs
`line_status_t`	Résultat de line_builder_put() et line_builder_end(). `LINE_NONE` : pas (encore) de ligne complète `LINE_READY` : une ligne complète est dans text[] `LINE_TOO_LONG` : la ligne dépassait CMD_LINE_MAX caractères : rejetée

Type	Description et valeurs
`line_builder_t`	Ligne en cours d'assemblage. `char text[CMD_LINE_MAX + 1U]` : caractères reçus, '\0' final quand prête `uint16_t len` : nombre de caractères dans text[] `bool too_long` : trop de caractères reçus pour cette ligne `bool done` : ligne rendue : la suivante repart de zéro

6.10 fifo : tampon circulaire

Fichiers `Core/Inc/fifo.h` (106 lignes) et `Core/Src/fifo.c` (93 lignes). Tampon circulaire d'octets (FIFO : « premier entré, premier sorti »), utilisé pour les messages en attente d'envoi vers le PC (USB) et vers le téléphone (Bluetooth).

Fonction	Rôle
`void fifo_init(fifo_t *f, uint8_t *storage, uint16_t size)`	Prépare un tampon circulaire (même rôle que FIFO_INIT, à l'exécution). `f` : tampon à préparer `storage` : tableau qui contiendra les octets `size` : taille du tableau en octets (puissance de 2, <= 32768)
`uint16_t fifo_used(const fifo_t *f)`	Nombre d'octets présents dans le tampon.
`uint16_t fifo_free(const fifo_t *f)`	Nombre d'octets qu'on peut encore écrire.
`bool fifo_write(fifo_t *f, const uint8_t *data, uint16_t len)`	Ajoute len octets à la fin du tampon, « tout ou rien » : s'il n'y a pas assez de place, rien n'est écrit (un message n'est jamais coupé en deux). `f` : tampon `data` : octets à ajouter `len` : nombre d'octets **Rend** : true si les octets ont été ajoutés, false si la place manquait
`uint16_t fifo_peek(const fifo_t *f, uint16_t offset, uint8_t *out, uint16_t len)`	Copie des octets du début du tampon sans les retirer (on regarde sans prendre ; fifo_drop() les retire ensuite). `f` : tampon `offset` : nombre d'octets à sauter depuis le début `out` : destination de la copie `len` : nombre d'octets souhaités **Rend** : nombre d'octets réellement copiés (moins de len si le tampon en contient moins)
`void fifo_drop(fifo_t *f, uint16_t len)`	Retire len octets du début du tampon (ou tous s'il y en a moins).
`void fifo_clear(fifo_t *f)`	Vide le tampon.

Type	Description et valeurs
`fifo_t`	Tampon circulaire. Les champs ne doivent être lus ou modifiés que par les fonctions fifo_xxx() ci-dessous. `uint8_t *buf` : tableau de stockage, fourni par l'utilisateur `uint16_t size` : taille du tableau : puissance de 2, <= 32768 `volatile uint16_t head` : compteur des octets écrits `volatile uint16_t tail` : compteur des octets retirés

6.11 fault : erreurs graves

Fichiers `Core/Inc/fault.h` (65 lignes) et `Core/Src/fault.c` (84 lignes). Signalement des erreurs graves par la LED rouge, pour une carte sans débogueur.

Fonction	Rôle
`void fault_init(void)`	Active les gestionnaires de faute détaillés (MemManage, BusFault, UsageFault) et le piège de la division entière par zéro. À appeler une fois au démarrage (début de app_init()).
`void fault_signal(fault_t cause)`	Arrête le programme et signale la cause par la LED rouge. Ne rend jamais la main. Appelée par Error_Handler() (main.c) et par les gestionnaires d'exception (stm32l4xx_it.c), dans leurs sections USER CODE. `cause` : cause de l'arrêt (nombre d'éclats)

Type	Description et valeurs

`fault_t`	Cause de l'arrêt : c'est aussi le nombre d'éclats de la LED rouge.
	`FAULT_ERROR_HANDLER`
	`FAULT_HARDFAULT`
	`FAULT_MEMMANAGE`
	`FAULT_BUSFAULT`
	`FAULT_USAGEFAULT`
	`FAULT_NMI`

6.12 hal_callbacks : rappels des interruptions

Fonction	Rôle
`void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`	appelée par la HAL après une interruption EXTI ; pour `USER_BUTTON_Pin`, appelle `button_on_interrupt()`
`void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)`	appelée par la HAL à chaque fin de période d'une minuterie ; pour TIM7, appelle `timer_1s_on_interrupt()`

6.13 Les réglages : app_config.h

Tous les nombres du programme qui peuvent se régler sont rassemblés dans `Core/Inc/app_config.h`, avec leur unité et la raison de leur valeur :

Réglage	Valeur	Signification
`APP_VERSION`	`"2.1.0"`	Version du programme (majeure.mineure.correctif). La version 2.0.0 reprend les commandes de OBCP_template_2024 et corrige ses défauts (voir README). La version 2.1.0 se charge par Bluetooth (chargeur ObCP-OTA) et ajoute la commande « #ota ».
`BLE_DEVICE_NAME`	`"OBCP-CROC"`	Nom de la carte vu par un téléphone en Bluetooth (paquet de publicité et caractéristique « Device Name »). 20 caractères au plus : le paquet de publicité ne fait que 31 octets. C'est le nom que cherchent les étudiants et l'application Android « ObCP Control » : ne le changer qu'en connaissance de cause.
`CMD_LINE_MAX`	`64U`	Longueur maximale d'une ligne de commande, en caractères (sans le '\0' final). La plus longue commande, « start timer », en compte 11 : 64 laisse de la marge pour de futures commandes. Une ligne plus longue est rejetée avec le message « False command ».
`CMD_VALUE_MAX`	`65535U`	Plus grande valeur acceptée par les commandes r, v, b et m (16 bits).
`USB_TX_FIFO_SIZE`	`1024U`	Taille du tampon d'émission USB, en octets. Il garde les messages en attente d'envoi vers le PC (l'aide, la plus longue réponse, fait environ 450 octets). Doit être une puissance de 2 (voir fifo.h).
`USB_TX_CHUNK_SIZE`	`256U`	Nombre maximal d'octets confiés en une fois au pilote USB. Le pilote les découpe lui-même en paquets de 64 octets (taille d'un paquet USB « full speed » pour la classe CDC).

Réglage	Valeur	Signification
BLE_CHUNK_SIZE	20U	Nombre maximal d'octets d'une notification BLE avec le MTU par défaut (23 octets, dont 3 d'en-tête ATT) : c'est la taille des morceaux envoyés au téléphone. Elle est aussi la taille des caractéristiques TX et RX.
BLE_TX_FIFO_SIZE	1024U	Taille du tampon d'émission BLE, en octets (messages en attente d'envoi, 2 octets de longueur en plus par message). Puissance de 2 (voir fifo.h).
BLE_RX_QUEUE_LEN	8U	Nombre de messages reçus du téléphone gardés en attente de traitement.
BLE_CHUNKS_PER_LOOP	4U	Nombre maximal de notifications envoyées à chaque tour de boucle : limite le temps passé dans l'envoi pour que l'USB et le bouton restent servis.
BLE_TX_RETRY_MS	20U	Attente, en ms, avant de redemander l'envoi d'une notification quand le module a répondu « plus de tampon libre » et n'a pas encore signalé qu'un tampon s'est libéré (évènement EVT_BLUE_GATT_TX_POOL_AVAILABLE).
BLE_ADV_RETRY_MS	200U	Attente, en ms, avant de relancer la publicité (« advertising ») si le module l'a refusée, par exemple juste après une déconnexion.
OTA_REPLY_TIMEOUT_MS	500U	Après « #ota » : temps maximal, en ms, laissé à la réponse pour partir vers le PC et le téléphone avant le redémarrage dans le chargeur. Il ne sert que si le PC ne lit plus le port COM (sinon, la réponse part en quelques millisecondes).
OTA_DISCONNECT_TIMEOUT_MS	500U	Après « #ota » : temps maximal, en ms, pour fermer proprement la connexion Bluetooth (et prévenir le PC : « BLE disconnected ») avant ce redémarrage. Au total, le passage au chargeur prend donc 1 s au plus.
BUTTON_DEBOUNCE_MS	30U	Durée, en ms, pendant laquelle le bouton doit rester stable pour qu'un appui (ou un relâchement) soit pris en compte : filtre les rebonds du contact mécanique, qui durent quelques millisecondes.
APP_SLEEP_WHEN_IDLE	1	1 : à la fin de chaque tour de boucle, le processeur s'endort (instruction WFI, « Wait For Interrupt ») jusqu'à la prochaine interruption (au plus 1 ms grâce au SysTick). La consommation baisse de quelques mA, ce qui compte sur batterie. 0 : la boucle tourne sans arrêt (comportement de la version 2024).

6.14 ota_switch : passage au chargeur Bluetooth

Fichiers `Core/Inc/ota_switch.h` (69 lignes) et `Core/Src/ota_switch.c` (108 lignes). Passage au chargeur Bluetooth (commande « #ota »), sans bloquer la boucle principale.

Fonction	Rôle
`void ota_switch_request(void)`	Demande le passage au chargeur Bluetooth (commande « #ota »). Le passage lui-même est fait par ota_switch_process(), dans la boucle principale. Une seconde demande pendant le passage est sans effet.
`void ota_switch_process(void)`	À appeler à chaque tour de la boucle principale : fait avancer le passage au chargeur s'il a été demandé (rien sinon). Ne bloque pas.
`bool ota_switch_is_pending(void)`	Indique si un passage au chargeur est en cours.

6.15 obcp_ota_request : demande au chargeur Bluetooth

Fichier `Core/Inc/obcp_ota_request.h` (92 lignes), sans fichier `.c` : deux fonctions `static inline`. Copie du fichier du projet ObCP_OTA_BLE (`ObCP_OTA_Demo/Core/Inc`), à garder identique. Boîte aux lettres : `OBCP_OTA_REQUEST_ADDRESS` (`0x10007FF0`) et `OBCP_OTA_REQUEST_MAGIC` (`0x4F544121`, « OTA! » en ASCII), les mêmes que dans le chargeur.

Fonction	Rôle
`void obcp_ota_init_vectors(void)`	Fait pointer le registre VTOR sur la table des vecteurs du programme (g_pfnVectors, à 0x08080000). À appeler au tout début de main(), dans USER CODE BEGIN 1. Après un reset, VTOR vaut 0 : la table du chargeur.

Fonction	Rôle
`void obcp_ota_enter_loader(void)`	Redémarre la carte dans le chargeur Bluetooth : dépose le mot magique et son complément dans la boîte aux lettres de la SRAM2, puis provoque un reset logiciel. Ne revient jamais.

7. Modifier le programme

7.1 Ajouter un traitement dans la boucle

La place prévue est l'étape 9 de `app_loop()`, dans `app.c` (commentaire « Votre code ici »). Le traitement doit être **court et sans attente**. Pour agir toutes les *N* millisecondes, on compare `HAL_GetTick()` à l'instant de la dernière action. L'exemple fourni (en commentaire) allume la LED bleue quand la carte est retournée :

```
static uint32_t last_ms;                    // dernière action (gardée d'un tour à l'autre)
if ((HAL_GetTick() - last_ms) >= 100U)      // 100 ms écoulées ?
{
  accel_g_t g;
  last_ms = HAL_GetTick();
  if (accel_read(&g))
  {
    outputs_set(OUTPUT_BLUE, (g.z < -0.5f) ? 20000U : 0U);
  }
}
```

`HAL_GetTick()` compte les millisecondes depuis le démarrage sur 32 bits : il repasse par 0 au bout de 49,7 jours. La soustraction `maintenant - dernière fois`, en entiers non signés, reste juste même à ce moment-là ; la comparaison `HAL_GetTick() >= last_ms + 100`, elle, se tromperait.

> **À éviter dans la boucle**
>
> `HAL_Delay()`, les boucles `while` qui attendent un évènement, les calculs très longs : pendant ce temps, l'USB et le Bluetooth ne sont plus servis.
>
> Les `printf()` « nus » : la sortie standard n'est reliée à rien. Utiliser `usb_console_printf()` ou `ble_uart_printf()`.

7.2 Ajouter une commande

Exemple : une commande `c` qui répond le nombre d'appuis sur le bouton (`Count = 3`).

1. dans `commands.h`, ajouter `CMD_COUNT` à l'énumération `cmd_id_t` (avant `CMD_BAD_VALUE`) ;
2. dans `commands_parse()`, reconnaître le texte, avec les autres commandes d'une lettre :

```
else if ((strcmp(text, "c") == 0) || (strcmp(text, "C") == 0))
{
  cmd.id = CMD_COUNT;
}
```

3. dans `commands_execute()`, ajouter le cas (le compteur `g_button_count` est une variable à créer dans `app.c`, augmentée à chaque appui à l'étape 7, déclarée `extern uint32_t g_button_count;` dans `app.h`, fichier à inclure dans `commands.c`) :

```
case CMD_COUNT:
  (void)snprintf(reply, sizeof(reply), "Count = %lu\n\r", (unsigned long)g_button_count);
  break;
```

4. compléter le texte d'aide `s_help` (une ligne terminée par `\n\r`) et le README ;
5. ajouter un test dans `Tests/test_unit.c` : `CHECK(commands_parse("c").id == CMD_COUNT);` puis `make` dans `Tests`.

7.3 Envoyer des données

Vers	Fonction	À savoir
le terminal du PC	`usb_console_printf("x = %d\n\r", x)`	127 caractères au plus par appel ; rend `false` si l'USB n'est pas prêt ou si la file est pleine
le téléphone	`ble_uart_printf("x = %d\n\r", x)`	un message = au moins une notification ; découpé par 20 octets ; rend `false` sans téléphone connecté

Vers	Fonction	À savoir
les deux	deux appels	c'est ce que fait `send_reply()` de `commands.c`

Pour afficher un nombre à virgule, `%f` fonctionne (option *Use float with printf* activée dans le projet) ; on convertit la variable `float` en `double` : `usb_console_printf("%.2f\n\r", (double)x)`. Un message Bluetooth de 20 caractères au plus part en une seule notification : c'est ce qui permet à l'application Android de reconnaître `X0.01Y-0.02Z1.00`.

7.4 Changer un réglage

Les réglages du § 6.13 se changent dans `app_config.h`, puis on recompile. Exemples : `BLE_DEVICE_NAME` (nom Bluetooth, 20 caractères au plus : le paquet de publicité ne fait que 31 octets), `BUTTON_DEBOUNCE_MS` (anti-rebond), `APP_SLEEP_WHEN_IDLE` (veille entre deux tours). Les tailles de file (`USB_TX_FIFO_SIZE`, `BLE_TX_FIFO_SIZE`) doivent rester des puissances de 2.

7.5 Modifier la configuration avec STM32CubeMX



Figure 14 – *modifier la configuration matérielle sans rien perdre*

1. ouvrir `OBCP_template_2026.ioc` (double-clic dans STM32CubeIDE) ; modifier les broches, minuteries, interruptions… ;
2. enregistrer (Ctrl + S) : CubeIDE propose de générer le code, répondre **Yes** ;
3. lancer l'outil de vérification depuis le dossier du projet : `python Tools/verif_cubemx.py` ;
4. compiler (*Project > Build Project*) et charger.

L'outil vérifie six choses et explique chaque erreur, avec la manière de la corriger : les crochets des sections USER CODE (§ 4.2) ; les réglages du `.ioc` dont dépend le programme (noms des broches, minuteries, interruptions, base de temps, options *Keep User Code* et fichiers `.c/.h` par périphérique) ; la présence dans `main.h` de chaque nom de broche utilisé par l'application ; la présence des fichiers de l'application ; l'unicité des fonctions de rappel de la HAL ; enfin, la place du programme derrière le chargeur Bluetooth (ligne `FLASH` du script `STM32L476RGTX_FLASH.ld` : en principe, CubeMX ne réécrit pas ce fichier, mais l'outil le contrôle quand même). Extrait de son compte rendu :

```
1. Crochets dans les sections USER CODE
  [OK]        Core/Src/main.c, section USER CODE 2 : app_init();
  ...
2. Réglages du fichier .ioc
  [ERREUR]    PC6.GPIO_Label = LD_ROUGE (attendu : LED_RED)
              -> dans CubeMX : clic droit sur la broche > Enter User Label
  [ATTENTION] TIM3.Period = 6399 (attendu : 65535)
  ...
Bilan : 1 erreur(s), 1 avertissement(s)
```

Pièges courants, tous détectés par l'outil :

- **renommer une broche** (*User Label*) : le code ne trouve plus `LED_RED_Pin`… ;
- **changer la base de temps de la HAL** (*System Core > SYS > Timebase Source*) : CubeMX écrit alors dans `main.c` un second `HAL_TIM_PeriodElapsedCallback()`, d'où une erreur « multiple definition » ;
- **décocher** l'option *Keep User Code when re-generating* : les crochets sont effacés ;
- **remettre le code par défaut de CubeMX** dans la section 6 de `usbd_cdc_if.c` (`USBD_CDC_ReceivePacket()`) : les paquets USB peuvent à nouveau s'écraser ;

- **retrouver la ligne FLASH d'origine** dans `STM32L476RGTX_FLASH.ld` (`ORIGIN = 0x8000000`) : le programme prendrait la place du chargeur Bluetooth (`obcp_ota.py` refuse alors de l'envoyer).

Ajouter une broche (une sortie pour une LED supplémentaire, par exemple) : clic sur la broche > *GPIO_Output*, clic droit > *Enter User Label* (`MY_LED`), générer ; `main.h` contient alors `MY_LED_Pin` et `MY_LED_GPIO_Port`, et `HAL_GPIO_WritePin(MY_LED_GPIO_Port, MY_LED_Pin, GPIO_PIN_SET)` l'allume. Le fichier `.ioc` a été enregistré avec STM32CubeMX 6.11.1 : une version plus récente propose de « migrer » le projet ; accepter, générer, puis lancer l'outil de vérification.

7.6 Déboguer

- Compiler en configuration **Debug** (`-O0 -g3`), brancher une sonde ST-LINK sur J17, puis *Run* > *Debug* : points d'arrêt, exécution pas à pas, valeur des variables au survol.
- La fenêtre **Live Expressions** suit des variables pendant que le programme tourne : par exemple `s_connected` et `s_conn_handle` (dans `ble_uart.c`), `s_state` (dans `button.c`).
- En cas de clignotement d'erreur grave (§ 3.7), mettre le programme en pause (*Suspend*) : il tourne dans `fault_signal()`. La fenêtre **Fault Analyzer** de STM32CubeIDE indique la faute et l'adresse de l'instruction en cause.
- La version Debug garde la sonde connectée pendant la veille `__WFI()` (fonction `HAL_DBGMCU_EnableDBGSleepMode()`, appelée seulement quand `DEBUG` est défini).
- Le programme étant rangé à `0x08080000`, la configuration de débogage ne doit pas effacer toute la flash avant de programmer : le chargeur Bluetooth, en banque 1, serait perdu. Grâce à `obcp_ota_init_vectors()` (§ 5.12), le programme fonctionne aussi lancé directement par le débogueur.

8. Compiler et tester

8.1 Avec STM32CubeIDE

File > Import > General > Existing Projects into Workspace, choisir le dossier du projet, *Finish* ; puis *Project > Build Project*. Deux configurations : **Debug** (`-O0 -g3`, pour le débogage pas à pas) et **Release** (`-Os`, programme optimisé, celui de `BIN/`). Les fichiers `.bin` et `.hex` sont produits à côté du `.elf`, dans le dossier `Debug/` ou `Release/` ; les trois contiennent le programme à l'adresse `0x08080000`. Le projet se compile sans aucun avertissement ; les fichiers de l'application passent aussi les options strictes `-Wall -Wextra -Wconversion -Wshadow -Wcast-qual` et l'analyseur statique cppcheck.

Le dossier `Tests/` est masqué dans CubeIDE par un filtre de ressources du projet : ses fichiers sont faits pour le PC, et ils auraient brouillé la navigation dans le code (*Open Declaration*). Pour les afficher : *Properties > Resource > Resource Filters*. Les fichiers du projet sont enregistrés en UTF-8 (réglage du projet, pour que les accents des commentaires s'affichent bien).

8.2 En ligne de commande

Le `Makefile` du projet compile sans STM32CubeIDE (Linux, macOS, WSL), avec la chaîne GNU Arm Embedded (`arm-none-eabi-gcc`) et les mêmes options que le projet CubeIDE : `make` (Release) ou `make DEBUG=1`, résultat dans `build/`.

8.3 Tests sur PC, sans carte

Le dossier `Tests/` compile le code de l'application **pour le PC**. Le matériel est remplacé par des **doublures** (*fakes*, dossier `Tests/fakes/`) : de fausses fonctions HAL qui simulent le temps, les broches et les minuteries, un faux module Bluetooth qui répond aux commandes ACI et peut simuler un téléphone, un faux PC sur l'USB (avec le contrôle de flux) et un faux accéléromètre (LIS3DH ou LIS3DSH). Le code testé est exactement celui de la carte : seules les fonctions de la HAL et des bibliothèques sont remplacées, à l'édition de liens.

```
cd Tests
make            # les trois séries
make unit       # tests unitaires
make app        # programme complet sur la carte simulée
make compare    # comparaison avec la version 2024
make demo2024   # démonstration d'un débordement de tableau de 2024
```

Série	Contenu	Résultat
`make unit`	tampon circulaire, assemblage des lignes, analyse de toutes les commandes (cas normaux et erreurs)	110 vérifications
`make app`	démarrage ; USB en paquets et au clavier, contrôle de flux, PC qui ne lit pas, déconnexion ; Bluetooth : connexion, commandes, aide découpée, recopie, module saturé, connexion échouée, publicité refusée, redémarrage du module ; bouton ; minuterie ; accéléromètre LIS3DH, LIS3DSH, absent, étalonnage ; commande #ota (USB et Bluetooth, réponse reçue avant le redémarrage, téléphone qui part aussitôt, PC qui ne lit plus, module qui ne confirme pas)	239 vérifications
`make compare`	le même scénario joué par la version 2024 (code d'origine) et la version 2026	61 étapes identiques

Les tests sont compilés avec **AddressSanitizer** et **UndefinedBehaviorSanitizer**, deux options de gcc qui arrêtent le programme au premier débordement de tableau, à la première lecture de mémoire libérée ou au premier calcul non défini (débordement d'entier signé…). Il faut `gcc`, `make` et `python3` (sous Windows : WSL ou MSYS2 ; sans les sanitizers : `make SAN=`). Pour ajouter un test, s'inspirer de `test_unit.c` : la macro `CHECK(condition)` compte les vérifications et affiche celles qui échouent.

Sur le PC, `obcp_ota_enter_loader()`, qui sur la carte redémarre le processeur, est remplacée par une fonction qui compte ses appels et revient (`Tests/fakes/obcp_ota_request.h`) : les tests de la commande `#ota` vérifient ainsi que la carte ne redémarre qu'une fois la réponse reçue par le PC et le téléphone.

8.4 Comparaison avec la version 2024

`Tests/compare/scenario.c` joue le même scénario sur les deux versions, compilées sur la même carte simulée : 61 étapes d'usage normal (commandes en paquets et au clavier, par USB et par Bluetooth, aide, effacement d'écran, minuterie, bouton, connexions, déconnexions), puis 21 cas qui révélaient des défauts. Chaque étape enregistre ce que la carte envoie au PC et au téléphone, et l'état des sorties PWM. `compare.py` compare les deux journaux : la partie commune doit être identique, aux mesures d'accélération près (la version 2024 avait une sensibilité fausse) et à la ligne #ota de l'aide près (ajoutée en version 2.1) ; les cas d'erreur sont affichés côte à côte (§ 10.2). `make demo2024` montre en plus que la version 2024 écrivait hors de son tableau `UsbBuffer2[10]` quand on tapait `start timer` touche par touche.

9. Exercices guidés

Ces exercices partent du programme fourni. Chacun se fait en quelques lignes ; les pistes indiquent les fonctions utiles. Penser à recompiler, puis à tester par le terminal et par le téléphone.

9.1 Un clignotement sans attente

Objectif : faire clignoter la LED verte (une seconde allumée, une seconde éteinte) sans jamais bloquer la boucle.

- à l'étape 9 de `app_loop()`, garder dans une variable `static` l'instant du dernier changement et l'état de la LED ;
- toutes les 1000 ms (`HAL_GetTick() - last_ms >= 1000U`), inverser l'état et appeler `outputs_set(OUTPUT_GREEN, …)` ;
- vérifier que les commandes répondent toujours instantanément : c'est la preuve que la boucle n'est pas bloquée. Comparer avec une version qui utiliserait `HAL_Delay(1000)`.

9.2 Une commande « c » : compteur d'appuis

Objectif : compter les appuis sur le bouton et répondre `Count = n` à la commande `c`.

- suivre les étapes du § 7.2 ; augmenter le compteur là où `button_poll_pressed()` rend `true` (étape 7 de `app_loop()`) ;
- ajouter la commande à l'aide et un test dans `Tests/test_unit.c` ;
- question : pourquoi le compteur ne doit-il pas être augmenté dans `HAL_GPIO_EXTI_Callback()` ? (penser aux rebonds, § 5.9).

9.3 Détecteur de chute

Objectif : envoyer `Chute !` au téléphone quand la carte tombe.

- en chute libre, l'accéléromètre mesure presque 0 g sur les trois axes ; calculer la norme $\sqrt{x^2 + y^2 + z^2}$ (fonction `sqrtf()` de `<math.h>`) toutes les 20 ms ;
- si elle reste sous 0,3 g pendant au moins 60 ms, envoyer le message avec `ble_uart_print()` (une seule fois par chute) ;
- la fréquence de mesure du capteur est de 10 par seconde (registre CTRL_REG1, § 5.8) : la monter à 100 par seconde (valeur 0x57) pour suivre une chute courte.

9.4 Changer le nom Bluetooth

Objectif : faire apparaître la carte sous le nom `OBCP-` suivi de vos initiales.

- modifier `BLE_DEVICE_NAME` dans `app_config.h`, recompiler, recharger ;
- le téléphone peut garder l'ancien nom en mémoire : désactiver puis réactiver son Bluetooth ;
- question : l'application ObCP Control trouve-t-elle encore la carte ? Que faudrait-il changer ?

9.5 Une broche de plus avec CubeMX

Objectif : commander une LED externe branchée sur PC4 (connecteur J8) par une commande `l1` / `l0`.

- dans CubeMX : PC4 en *GPIO_Output*, nom `MY_LED` ; générer le code ;
- lancer `python Tools/verif_cubemx.py` : le bilan doit rester à 0 erreur ;
- ajouter la commande (§ 7.2) avec `HAL_GPIO_WritePin(MY_LED_GPIO_Port, MY_LED_Pin, …)`, ou mieux, une fonction dans `outputs.c`.

10. De la version 2024 à la version 2026

10.1 Ce qui ne change pas

Les commandes et leurs réponses, les messages spontanés, le nom Bluetooth, le service et ses UUID (y compris l'inversion de TX et RX), l'adresse Bluetooth, le code d'appairage, les broches et la sortie moteur. La comparaison automatique (§ 8.4) le vérifie sur 61 étapes d'usage normal. La version 2.1 ajoute seulement la commande #ota et sa ligne dans l'aide.

10.2 Ce que l'on peut voir

Les différences visibles ne concernent que des cas d'erreur, et l'échelle des LED (§ 10.4) :

Situation	Version 2024	Version 2026
`r5` sans fin de ligne juste après `r65535`	`Red = 55535`	`Red = 5`
`r5` écrit par le téléphone (tampon de la bibliothèque réutilisé)	`Red = 55535`	`Red = 5`
`b9` tapé au clavier, Entrée = CR+LF	`False command`	`Blue = 9`
`start timer` tapé touche par touche	débordement du tableau `UsbBuffer2[10]`	minuterie lancée
`r010`	`Red = 8` (lu en octal)	`Red = 10`
`r-1`	`Red = -1`, LED à fond	`False value (0-65535)`
`r70000`	`Red = 70000` ; le registre 16 bits reçoit 4464	`False value (0-65535)`
`r` seul	`Red = 0`	`False value (0-65535)`
`r50x`	`Red = 50`	`False value (0-65535)`
`blue`	`Blue = 0`	`False command`
ligne vide (Entrée seule)	`False command`	rien
`a`, carte posée à plat	`Z=0.96` (4 % trop faible)	`Z=1.00`
`start timer`, `stop timer`, `#rst`	une notification Bluetooth vide	rien
accéléromètre absent	valeurs sans signification	`Accelerometer error`
`rn` avec *n* > 6399	LED à pleine lumière dès 6399	réglage progressif jusqu'à 65535

10.3 Défauts corrigés

Domaine	Défaut de la version 2024	Correction
USB	paquet reçu lu sans `'\0'` final, dans un tampon réutilisé : restes de la commande précédente	copie terminée par `'\0'` (`usb_console.c`)
USB	frappe au clavier : tableau de 10 cases sans contrôle de place, débordement dès 11 caractères	ligne limitée à 64 caractères, rejet propre (`line_builder.c`)
USB	seul CR terminait une ligne ; le LF d'un Entrée CR+LF restait au début de la commande suivante	CR, LF et CR+LF acceptés ; effacement géré
USB	paquet suivant autorisé avant la fin du traitement : écrasement possible	contrôle de flux (§ 5.5)
USB	`CDC_Transmit_FS()` appelée directement : message perdu si l'envoi précédent n'était pas fini ; tableau local envoyé après la fin de la fonction	file d'envoi de 1 ko ; `CDC_Transmit_FS()` passe par elle
BLE	texte reçu lu au-delà de sa longueur, dans un tampon réutilisé	copie terminée par `'\0'` (`ble_uart.c`)
BLE	connexion échouée prise pour une connexion réussie : plus de publicité jusqu'au reset	statut vérifié, publicité relancée
BLE	publicité refusée (juste après une déconnexion) jamais réessayée	nouvel essai toutes les 200 ms

Domaine	Défaut de la version 2024	Correction
BLE	redémarrage inattendu du module non traité : service perdu	module remis en service
BLE	commandes exécutées dans la fonction d'évènement du module, avec des attentes actives	exécution dans la boucle, envoi sans attente
BLE	morceau de notification abandonné après 10 ms si le module était saturé	nouvel essai quand un tampon se libère
BLE	notification vide après `start timer`, `#rst`… et après un message d'exactement 20 octets	supprimée
BLE	risque de blocage si les 5 tampons de la bibliothèque sont pleins (IRQ restée à 1)	lecture relancée par la boucle
BLE	adresse lue sans contrôle : au premier démarrage d'un module neuf, adresse vide	adresse vérifiée, adresse de secours propre à chaque STM32
BLE	caractéristique *Device Name* laissée à « BlueNRG » ; résultats des commandes ACI jamais vérifiés	« OBCP-CROC » ; chaque étape vérifiée, panne signalée par la LED
Bouton	anti-rebond par attentes : boucle bloquée tant que le bouton est enfoncé	machine à états sans attente (`button.c`)
Minuterie	`Prescaler = 6400` : période de 1,0001 s ; toute minuterie déclenchait les mesures ; première seconde incomplète après un arrêt	6399 : 1 s exactement ; seule TIM7 compte ; comptage repris à zéro
Accéléromètre	sensibilité du LIS3DSH appliquée au LIS3DH (4 % d'erreur) ; trois lectures séparées des axes ; copie de la structure SPI ; présence jamais vérifiée	sensibilité du circuit détecté ; lecture des 6 octets d'un coup ; pointeur vers `hspi3` ; registre d'identité lu
Commandes	`sscanf("%c%i")` : octal, négatif, trop grand, valeur absente ou suivie d'autre chose acceptés ; comparaison du début seulement (`start timerXYZ`) ; aide « b50 for red »	analyse stricte, `False value (0-65535)` ; comparaison exacte ; « b50 for blue »
Sorties	TIM3 jusqu'à 6399 alors que les commandes vont jusqu'à 65535 ; TIM1 sans *preload*	TIM3 jusqu'à 65535 ; *preload* activé
Broches	PB2 (sélection du module) nommée `BLE_Reset_Line` et à 0 au démarrage ; PC3 (sélection de l'EEPROM) non configurée	`BLE_CS` à 1 ; `EEPROM_CS` en sortie à 1
Système	tas de 512 octets et pile de 1 ko ; erreur grave silencieuse ; processeur toujours à pleine vitesse	2 ko et 4 ko ; code de la LED rouge ; veille `__WFI()`
Code	en-tête `includes.h` inclus partout, variables globales partagées, logique dans les interruptions	modules indépendants, interface `.h` documentée

10.4 Un choix à valider : l'échelle des LED

En 2024, TIM3 comptait de 0 à 6399 (avec un *Prescaler* de 9, soit une PWM à 1 kHz) : au-delà de 6399, la LED était déjà à pleine lumière, alors que l'aide, les curseurs des scripts Python (0 à 65535) et l'application ObCP Control (0 à 65535, avec une courbe « perceptive ») utilisent toute l'échelle 16 bits. La version 2026 fait compter TIM3 de 0 à 65535 (PWM à 976,6 Hz, fréquence presque inchangée) : la réponse est la même, mais **une même valeur éclaire moins** (`r6399` donne maintenant un rapport cyclique de 10 % au lieu de 100 %).

Si un sujet de TP ou le VI LabVIEW suppose l'échelle 0 à 6399, remettre dans CubeMX *TIM3 > Prescaler = 9, Counter Period = 6399* puis générer : l'outil de vérification affichera un simple avertissement. La sortie moteur (TIM1) n'a pas changé.

10.5 Correspondance des noms

Version 2024	Version 2026
`processCommand()`, `USBCmd()`, `BLE_command_received()` (Driver_OBCP.c)	`commands_parse()`, `commands_execute()` (commands.c)
`receiveDataUSB()`, `UsbBuffer`, `UsbBuffer2`, `RxUSB_OK`	`usb_console_read_line()` (usb_console.c), `line_builder.c`
`CDC_Transmit_FS()`	`usb_console_print()`, `usb_console_printf()` ; `CDC_Transmit_FS()` reste utilisable
`Printf_BLE()`, `sendDataDef()`	`ble_uart_print()`, `ble_uart_printf()`
`Bluetooh_LE_Process()`, `User_Process()`	`app_loop()` (app.c)

Version 2024	Version 2026
`reset_OBCP_functions()`	`outputs_all_off()` et `timer_1s_stop()`
`__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, v)`	`outputs_set(OUTPUT_RED, v)`
`user_button_pressed`, `Acc_timer`	`button_poll_pressed()`, `timer_1s_take_tick()`
`HAL_GPIO_EXTI_Callback()`, `HAL_TIM_PeriodElapsedCallback()` (Driver_OBCP.c)	hal_callbacks.c
`MX_BlueNRG_MS_Init()`, `Add_UART_BLE_Service()`, `Make_Connection()` (BLE_service.c)	`ble_uart_init()` (ble_uart.c)
`user_notify()`, `Attribute_Modified_CB()`, `GAP_ConnectionComplete_CB()`…	`ble_event_handler()` (ble_uart.c)
`BLE_connected`, `USB_Connected`	`ble_uart_is_connected()`, `usb_console_is_ready()`
`LIS3DSH_Init()`, `LIS3DSH_GetDataScaled()`, `LIS3DSH_GetTemperature()`, `LIS3DSH_X_calibrate()`	`accel_init()`, `accel_read()`, `accel_read_temperature()`, `accel_set_calibration()`
étiquettes `B1`, `BLE_Reset`, `BLE_Reset_Line`, `BLE_EXTI_Line`, `PWM_Motor`, `Led_Red`…	`USER_BUTTON`, `BLE_RST`, `BLE_CS`, `BLE_IRQ`, `PWM_MOTOR`, `LED_RED`… (+ `EEPROM_CS`)

Annexe A. Glossaire

Terme	Définition
ACI	*Application Command Interface* : jeu de commandes que le STM32 envoie au module BlueNRG (fonctions `aci_…` de la bibliothèque).
anti-rebond	traitement qui ignore les ouvertures et fermetures parasites d'un contact mécanique pendant quelques millisecondes.
BLE	*Bluetooth Low Energy* : Bluetooth à basse consommation, utilisé par les objets connectés et les téléphones.
CDC	*Communication Device Class* : classe USB qui fait voir un appareil comme un port série (port COM virtuel).
chargeur	petit programme rangé au début de la flash, qui démarre en premier et peut recevoir un nouveau programme (*bootloader*) ; ici, le chargeur Bluetooth ObCP-OTA (banque 1), à côté du chargeur DFU intégré au STM32.
CubeMX	outil de ST qui génère le code de configuration du microcontrôleur à partir du fichier `.ioc`.
DFU	*Device Firmware Upgrade* : chargement d'un programme par l'USB, grâce au chargeur intégré au STM32 (interrupteur SW4 sur « Prog »).
drapeau	variable booléenne levée par une interruption pour signaler un évènement à la boucle principale.
EXTI	contrôleur d'interruptions externes : déclenche une interruption sur un front d'une broche.
FIFO	*First In, First Out* : file d'attente, ici sous forme de tampon circulaire (`fifo.c`).
GAP, GATT	couches du Bluetooth : GAP gère la publicité et les connexions, GATT les services et caractéristiques.
HAL	*Hardware Abstraction Layer* : bibliothèque de ST qui donne accès aux périphériques par des fonctions (`HAL_GPIO_WritePin()`…).
HCI	*Host Controller Interface* : couche de transport entre le STM32 et le module Bluetooth (ici par SPI).
interruption	évènement matériel qui suspend le programme pour exécuter une fonction courte (gestionnaire), puis reprend.
NAK	réponse « pas prêt » d'un appareil USB : l'ordinateur réessaie plus tard (contrôle de flux).
notification	envoi spontané, par la carte, de la nouvelle valeur d'une caractéristique au téléphone abonné.
OTA	*Over The Air* : chargement d'un programme sans fil, ici par Bluetooth (commande `#ota`).
publicité (advertising)	paquets émis régulièrement par un appareil BLE pour se faire connaître.
PWM	*Pulse Width Modulation* : signal carré dont on règle la fraction du temps à 1 (rapport cyclique).
rapport cyclique	fraction de la période pendant laquelle un signal PWM est à 1, de 0 à 100 %.
SPI	bus série synchrone à quatre fils (SCK, MOSI, MISO, CS) entre un maître et des esclaves.
super boucle	organisation d'un programme sans système d'exploitation : une boucle sans fin qui sert chaque tâche à tour de rôle.
UUID	identifiant unique de 128 bits d'un service ou d'une caractéristique Bluetooth.
USER CODE	sections des fichiers générés, entre `/* USER CODE BEGIN xxx */` et `/* USER CODE END xxx */`, que CubeMX conserve.
`volatile`	mot du C qui oblige le compilateur à relire une variable en mémoire à chaque usage (variable modifiée par une interruption).
VTOR	*Vector Table Offset Register* : registre du processeur qui indique où se trouve la table des interruptions (**`obcp_ota_init_vectors()`**).
WFI	*Wait For Interrupt* : instruction qui endort le processeur jusqu'à la prochaine interruption.

Annexe B. Documentation et liens

Sujet	Document
procédure de programmation de l'ObCP	`Datasheets/Procedure_Programmation_OBCP_CROC.pdf`

Sujet	Document
module Bluetooth	`Datasheets/bluenrg-m0.pdf`, `SPBTLE-RFen.DM00381960.pdf`
programmation BlueNRG-MS, commandes ACI	`Datasheets/pm0237-….pdf` (PM0237), `um1865-….pdf` (UM1865)
adresse Bluetooth du module	`Datasheets/dt0070-….pdf` (DT0070)
accéléromètre LIS3DH	`Datasheets/lis3dh.pdf` ; https://www.st.com/resource/en/datasheet/lis3dh.pdf
minuteries et PWM	https://wiki.st.com/stm32mcu/wiki/Getting_started_with_TIM
interruptions externes	https://wiki.st.com/stm32mcu/wiki/Getting_started_with_EXTI
bibliothèque USB de ST (UM1734)	https://www.st.com/resource/en/user_manual/um1734-stm32cube-usb-device-library-stmicroelectronics.pdf
processeur Cortex-M4, fautes (PM0214)	https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf
service UART de Nordic (NUS)	https://developer.nordicsemi.com/nRF_Connect_SDK/doc-legacy/2.6.1/nrf/libraries/bluetooth_services/services/nus.html
STM32CubeProgrammer	https://www.st.com/en/development-tools/stm32cubeprog.html
chargement par Bluetooth	projet ObCP_OTA_BLE : chargeur, application PC obcp_ota.py, guide ObCP_OTA_BLE_Guide.docx
dépôt de l'ObCP, applications Android	https://github.com/jimbaud/LAB_ObCP_L476_STM32CubeIDE

Annexe C. Versions des outils

Outil	Version
STM32CubeMX (fichier `.ioc`)	6.11.1
paquet STM32Cube FW_L4	1.18.1
paquet X-CUBE-BLE1 (bibliothèque BlueNRG-MS)	6.2.3
compilateur	GNU Arm Embedded (celui de STM32CubeIDE) ; langage C11 avec extensions GNU
programme	OBCP_template_2026 2.1.0
chargeur Bluetooth (projet ObCP_OTA_BLE)	ObCP_BLE_Loader 1.1